

Chapter 8

Security

A note on the

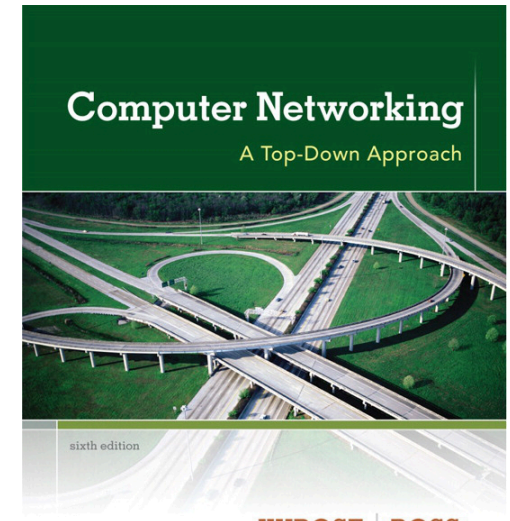
We're making these

They're in PowerPoint form so you see the animations; and can add, modify, and delete slides (including this one) and slide content to suit your needs. They obviously represent a lot of work on our part. In return for use, we only ask the following:

- ❖ If you use these slides (e.g., in a class) that you mention their source (after all, we'd like people to use our book!)
- ❖ If you post any slides on a www site, that you note that they are adapted from (or perhaps identical to) our slides, and note our copyright of this material.

Thanks and enjoy! JFK/KWR

©All material copyright 1996-2012
J.F Kurose and K.W. Ross, All Rights Reserved



orking: A
Top Down
Approach
6th edition
Jim Kurose, Keith
Ross
Addison-Wesley
March 2012

Chapter 8: Network Security

Chapter goals:

- ❖ understand principles of network security:
 - cryptography ses beyond
 - message integrity
- ❖ security in practice:
 - firewalls and intrusion detection systems
 - security in application, transport, network, link layers

Chapter 8 roadmap

8.1 What is network security?

8.2 Principles of cryptography

8.5 Securing TCP connections: SSL

8.6 Network layer security: IPsec

8.7 Securing wireless LANs

8.8 Operational security: firewalls and
IDS

What is network security?

confidentiality: only sender, intended receiver should “understand” message contents

- sender encry

authen

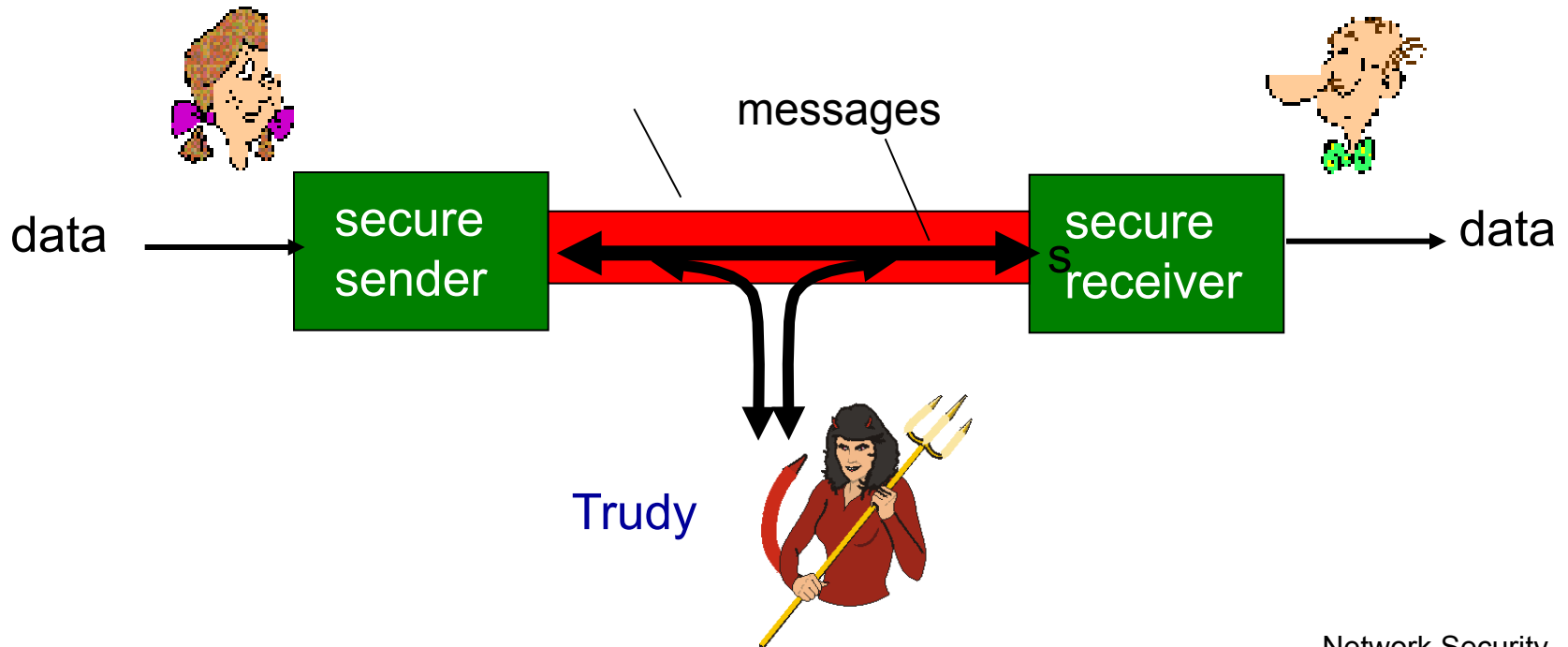
confirm identity of each other

message integrity: sender, receiver want to ensure message not altered (in transit, or afterwards) without detection

access and availability: services must be accessible and available to users

Friends and enemies: Alice, Bob, Trudy

- ❖ well-known in network security world
- ❖ Bob, Alice (lovers!) want to communicate “securely”
- ❖ Trudy (intruder delete, add



Who might Bob, Alice be?

- ❖ ... well, *real-life* Bobs and Alices!
- ❖ Web browser/server for electronic transactions (purchases)
- ❖ DNS
- ❖ routers exchanging routing table updates
- ❖ other examples?

There are bad guys (and girls) ~~out there!~~

Q: What can a “bad guy” do?

A: A lot! See section 1.6

- *eavesdrop*: intercept messages
- *im*
source address in packet (or any field in packet)
- *hijacking*: “take over” ongoing connection by removing sender or receiver, inserting himself in place
- *denial of service*: prevent service from being used by others (e.g., by

Chapter 8 roadmap

8.1 What is network security?

8.2 Principles of cryptography

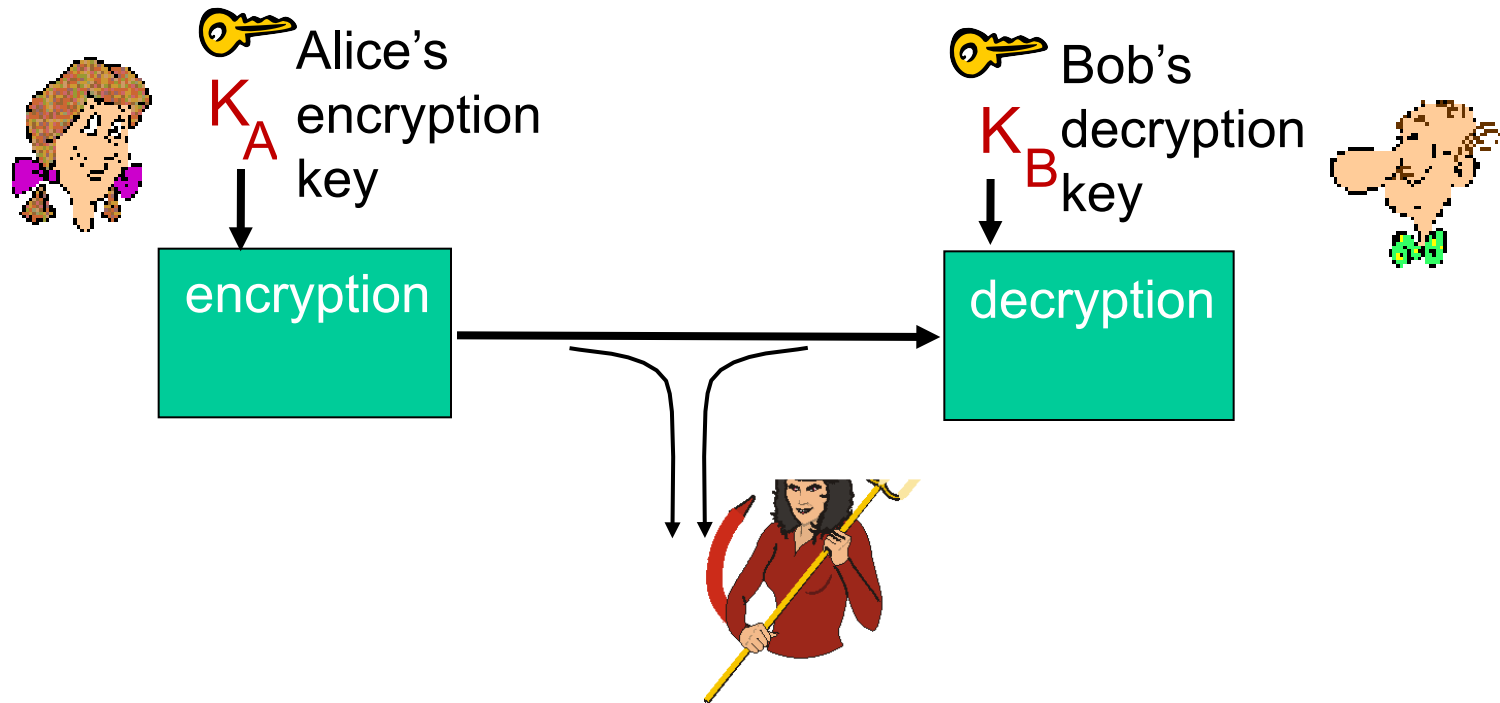
8.5 Securing TCP connections: SSL

8.6 Network layer security: IPsec

8.7 Securing wireless LANs

8.8 Operational security: firewalls and
IDS

The language of cryptography



m plaintext message

$K_A(m)$ ciphertext, encrypted with key K_A

$m = K_B(K_A(m))$

Breaking an encryption scheme

❖ **cipher-text only attack:** Trudy has ciphertext she

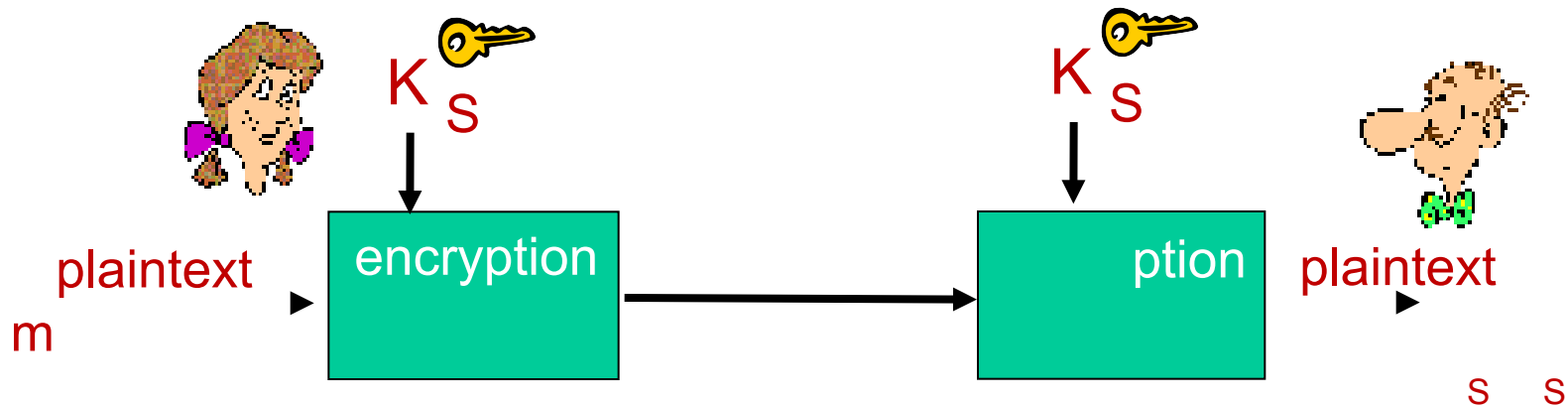
- brute force search through all keys
- statistical analysis

❖ **known-plaintext attack:** Trudy has plaintext

monoalphabetic cipher, Trudy determines pairings for a, l, i, c, e, b, o,

❖ **chosen-plaintext attack:** Trudy can get ciphertext for chosen plaintext

Symmetric key cryptography



symmetric key crypto: Bob and Alice share same (symmetric) key: K

- ❖ e.g., key is knowing substitution pattern in mono alphabetic substitution cipher

Q: how do Bob and Alice agree on key value?

Simple encryption scheme

substitution cipher: substituting one thing for another

- monoalphabetic cipher: substitute one letter for another

plaintext:		n o p q r s t u v w x y z
	↓	↓
ciph		t r e w q

e.g.: Plaintext: bob. i love you. alice
ciphertext: nkn. s gktc wky. mgsbc

🔑 *Encryption key*: mapping from set of 26 letters
to set of 26 letters

A more sophisticated encryption approach

- ❖ n substitution ciphers, $M_1, M_2, \dots, M_n$
- ❖ cycling pattern:
 - e.g., $n=4$: M_1, M_3, M_4, M_3, M_2 ; M_1, M_3, M_4, M_3, M_2 ;
 - ..

subs
pattern

yclic

- 🔑 ▪ dog: d from M_1 , o from M_3 , g from M_4

Encryption key: n substitution ciphers,
and cyclic pattern

- key need not be just n-bit pattern

Symmetric key crypto: DES

DES: Data Encryption Standard

- ❖ US encryption standard [NIST 1993]
- ❖ 56-bit symmetric key, 64-bit plaintext input
- ❖ block cipher with chaining
 - DES phrase decrypted (brute force) in less than a day
 - no known good analytic attack
- ❖ making DES more secure:
 - 3DES: encrypt 3 times with 3 different keys

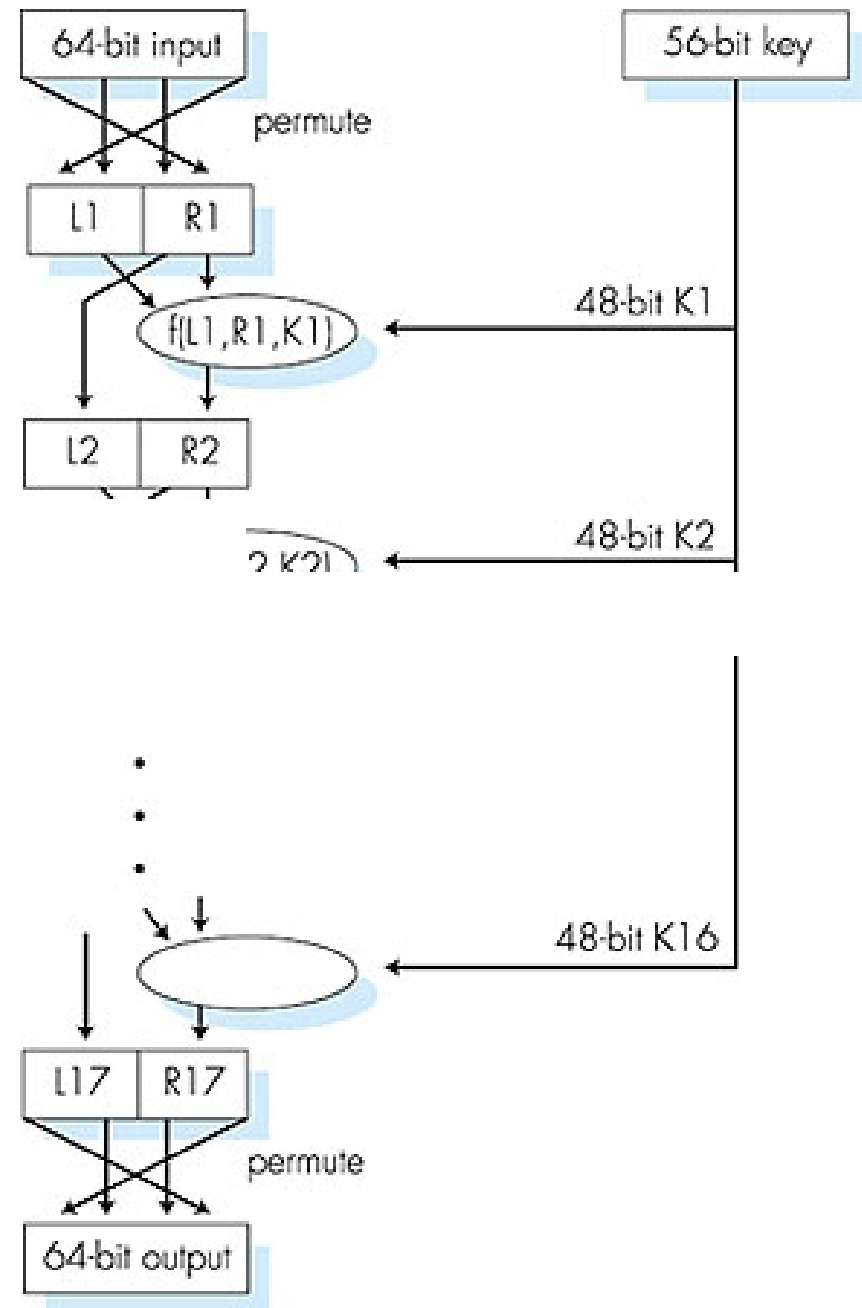
Symmetric key crypto: DES

DES operation

initial permutatio

of fun
application, each
using different 48
bits of key

final permutation



AES: Advanced Encryption Standard

- ❖ symmetric-key NIST standard, replaced DES (Nov 2001)
- ❖
- ❖ brute force
taking 1 sec on DES, takes 149 trillion years for AES

Public Key Cryptography



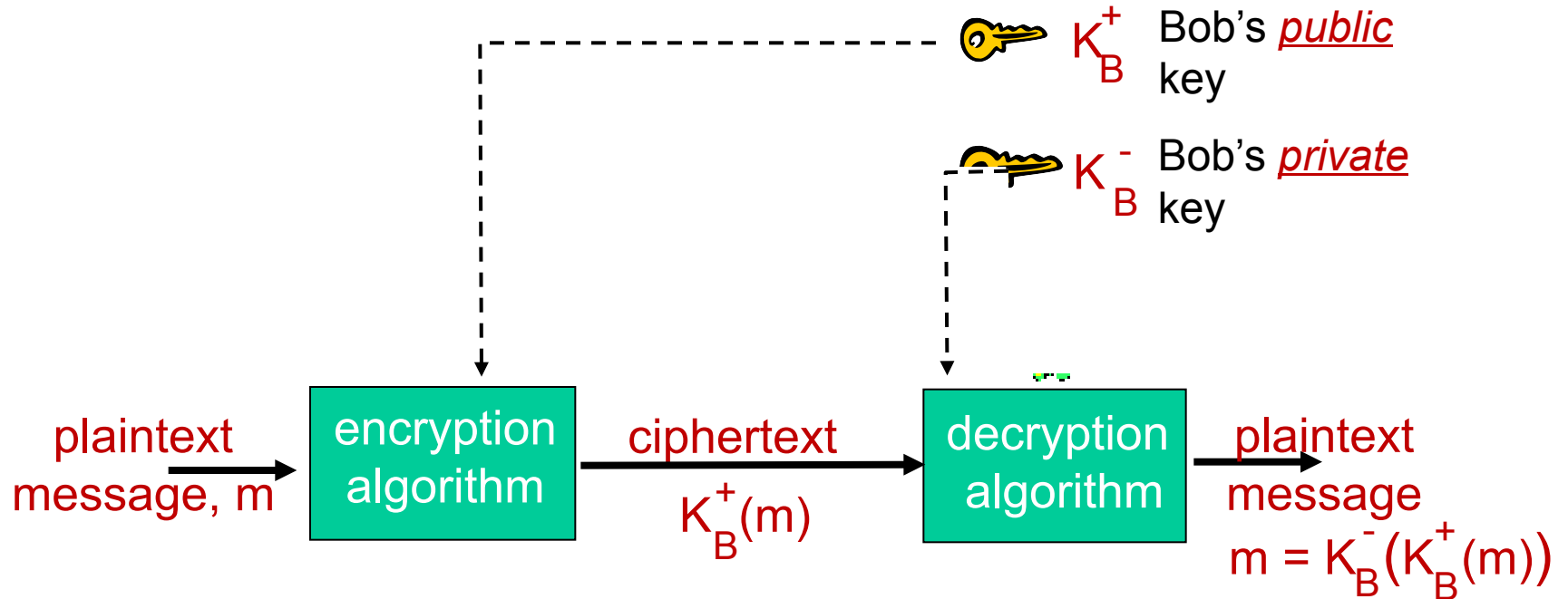
symmetric key crypto

- ❖ requires sender, receiver to share a secret key
- ❖ Q: how to agree on key in first place (particularly if never "met")?

public key crypto

- ❖ radically different approach [Diffie-Hellman]
- ❖ sender does *not* share secret key
- ❖ *public* encryption key known to *all*
- ❖ *private* decryption key known only to receiver

Public key cryptography



Public key encryption algorithms

requirements:

① need K^+ - ch that

② given public key K_B^+ , it should be impossible to compute private key K_B^-

RSA: Rivest, Shamir, Adelson algorithm

Prerequisite: modular arithmetic

- ❖ $x \bmod n$ = remainder of x when divide by n
- ❖ facts:

$$[(a \bmod n) * (b \bmod n)] \bmod n = (a*b) \bmod n$$

- ❖ thus

$$(a \bmod n)^d \bmod n = a^d \bmod n$$

- ❖ example: $x=14, n=10, d=2$:
 $(x \bmod n)^d \bmod n = 4^2 \bmod 10 = 6$
 $x^d = 14^2 = 196 \quad x^d \bmod 10 = 6$

RSA: getting ready

- ❖ message: just a bit pattern
- ❖ bit pattern can be uniquely represented by an integer number

example:

- ❖ $m = 10010001$. This message is uniquely represented by the decimal number 145.
- ❖ to encrypt m , we encrypt the corresponding number, which gives a new number (the ciphertext).

RSA: Creating public/private key pair

1. choose two large prime numbers p, q .
(e.g., 1024 bits each)

2. compute $n = p \cdot q$ and $\phi(n) = (p-1)(q-1)$

wit

4. choose d such that $ed-1$ is exactly divisible
(in other words: $ed \bmod \phi(n) = 1$).

5. *public* key is (n, e) . *private* key is (n, d) .
 $\underbrace{(n, e)}_{K_B^+} \quad \underbrace{(n, d)}_{K_B^-}$

RSA: encryption, ~~decryption~~

0. given (n, e) and (n, d) as computed above

1. to encrypt message $m (< n)$, compute

2. to compute

$$m^d = c \pmod n$$

magic happens!

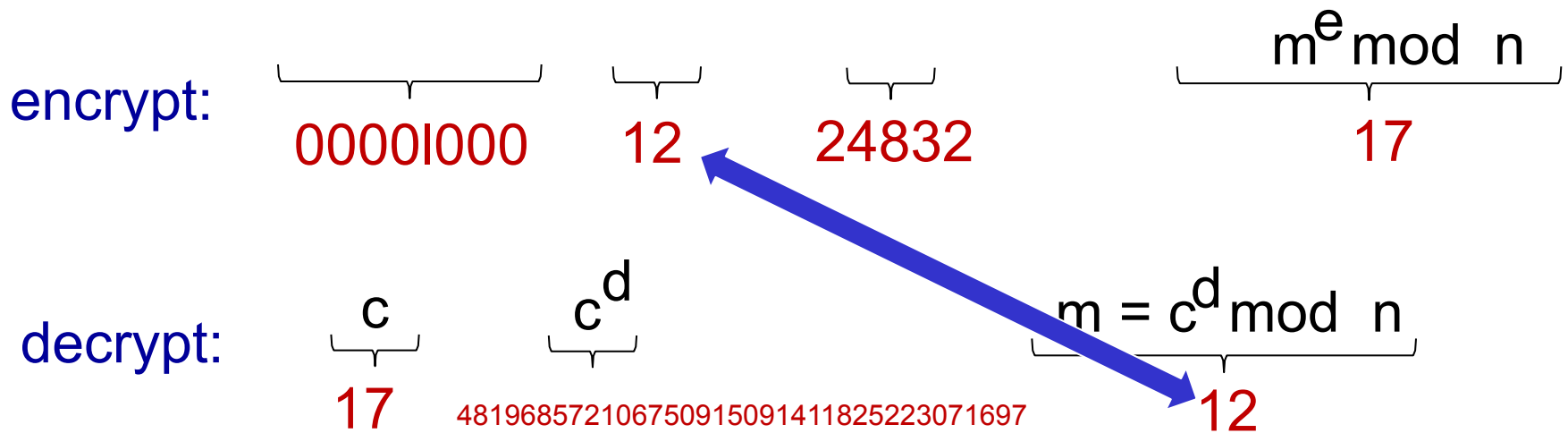
$$m = \underbrace{(m^e \pmod n)_c}_c^d \pmod n$$

RSA example:

Bob chooses $p=5$, $q=7$. Then $n=35$, $z=24$.

$e=5$ (so e , z relatively prime).

$d=29$ (so $ed-1$ exactly divisible by z).



Why does RSA work?

❖ must show that $c^d \bmod n = m$
where $c = m^e \bmod n$

❖ fact: for any x $= x^{(y \bmod z)} \bmod$

❖ thus,

$$\begin{aligned} c^d \bmod n &= (m^e \bmod n)^d \bmod n \\ &= m^{ed} \bmod n \\ &= m^{(ed \bmod z)} \bmod n \\ &= m^1 \bmod n \\ &= m \end{aligned}$$

RSA: another important ~~property~~

The following property will be *very* useful later:

$$K^{-}(K^{+}(m)) = K^{-}(K^{+}(K^{-}(m)))$$

us

first, followed
by private key

key first,
followed by
public key

*result is the
same!*

Why $K_B^-(K_B^+(m)) = m = K_B^+(K_B^-(m))$?

follows directly from modular arithmetic:

$$= (m^d \bmod n)^e \bmod n$$

Why is RSA secure?

- ❖ suppose you know Bob's public key (n, e) . How hard is it to determine d ?
- ❖ essentially no known algorithm for finding factors of n
 - fact: factoring a big number is hard

RSA in practice: session keys

- ❖ exponentiation in RSA is computationally intensive
- ❖ DES is at least 100 times faster than RSA
- ❖ use RSA to establish first secure connection, then establish second key – symmetric session key – for encrypting data

session key, K_S

- ❖ Bob and Alice use RSA to exchange a symmetric key K_S
- ❖ once both have K_S , they use symmetric key

Chapter 8 roadmap

8.1 What is network security?

8.2 Principles of cryptography

8.5 Securing TCP connections: SSL

8.6 Network layer security: IPsec

8.7 Securing wireless LANs

8.8 Operational security: firewalls and
IDS

Authentication

Goal: Bob wants Alice to “prove” her identity to him

protocol ap1.0: Alice “Alice”



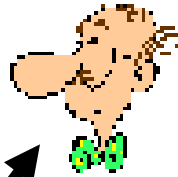
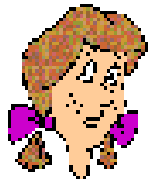
Failure scenario??



Authentication

Goal: Bob wants Alice to “prove” her identity to him

protocol ap1.0: Alice “Alice”

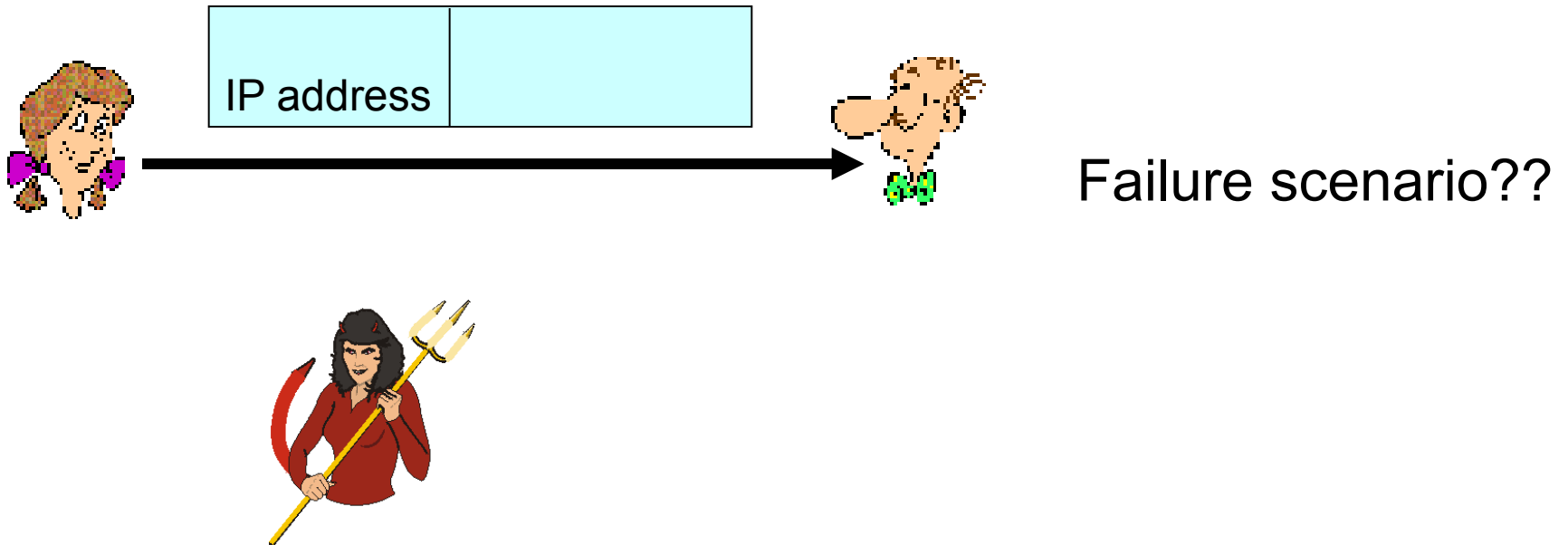


“I am Alice”

in a network,
Bob can not “see” Alice,
so Trudy simply declares
herself to be Alice

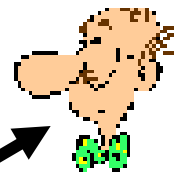
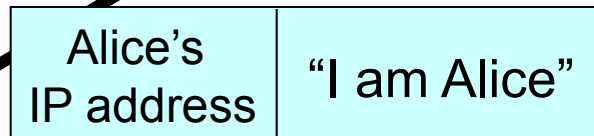
Authentication: another try

Protocol ap2.0: Alice says “I am Alice” in an IP packet containing her source IP address



Authentication: another try

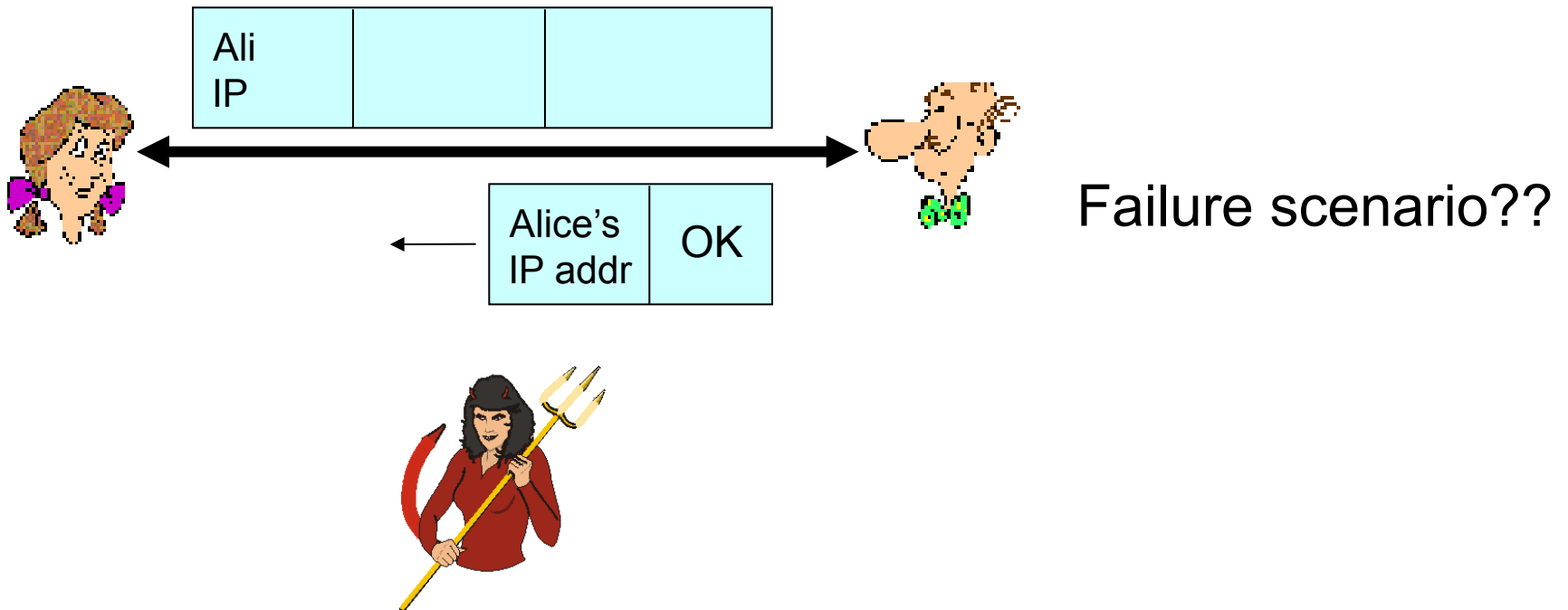
Protocol ap2.0: Alice says “I am Alice” in an IP packet containing her source IP address



Trudy can create a packet “spoofing” Alice’s address

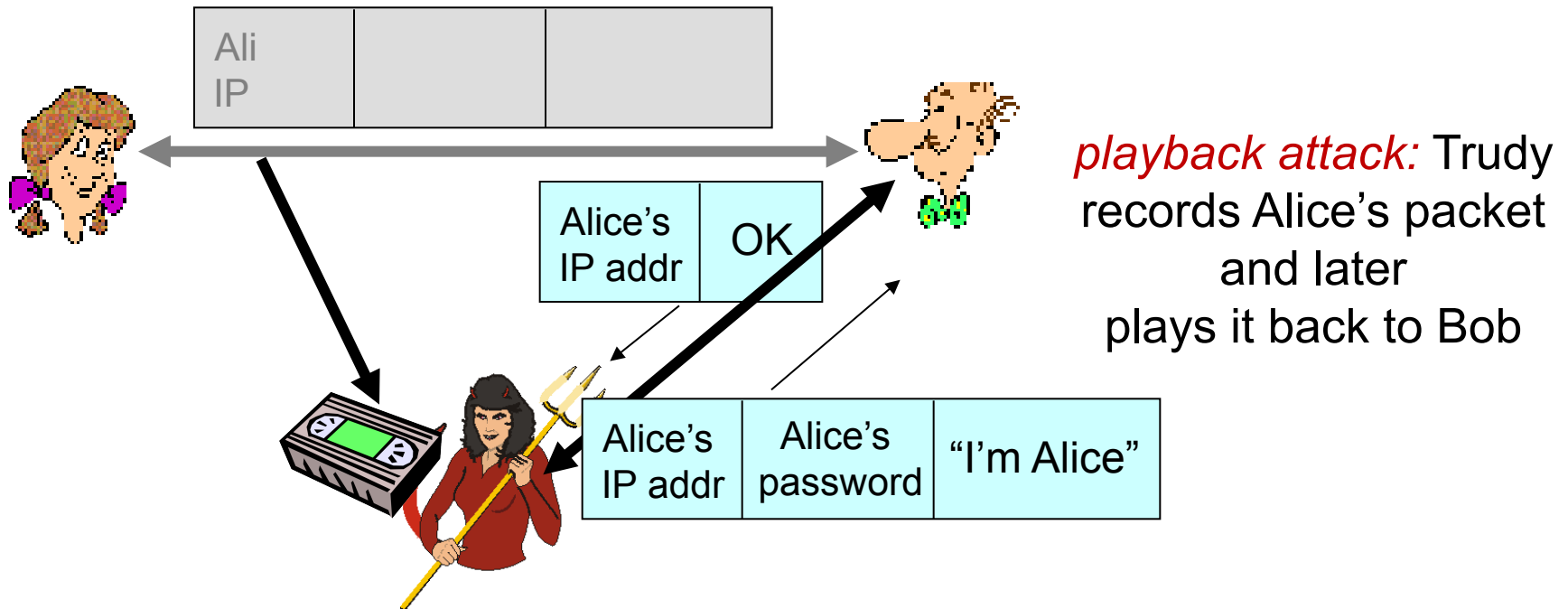
Authentication: another try

ol ap3.0: Alice says "I am Alice" and sends her secret password to "prove" it.



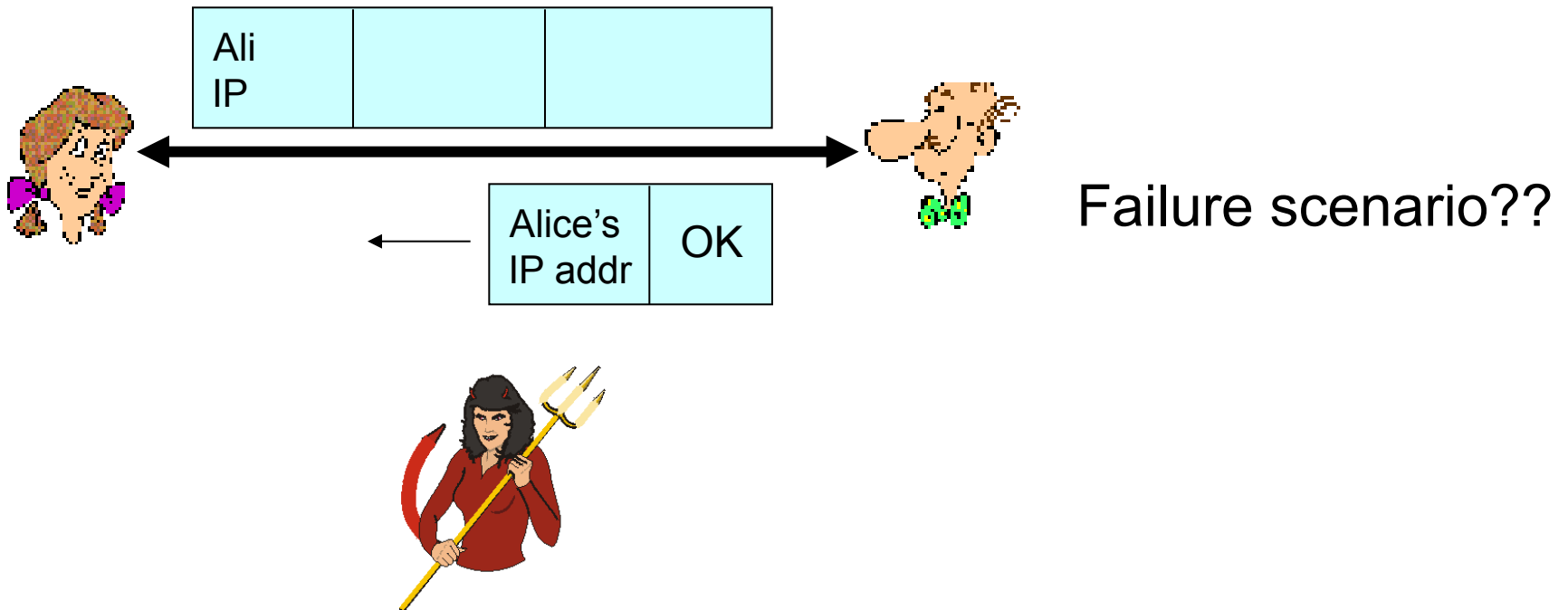
Authentication: another try

ol ap3.0: Alice says "I am Alice" and sends her secret password to "prove" it.



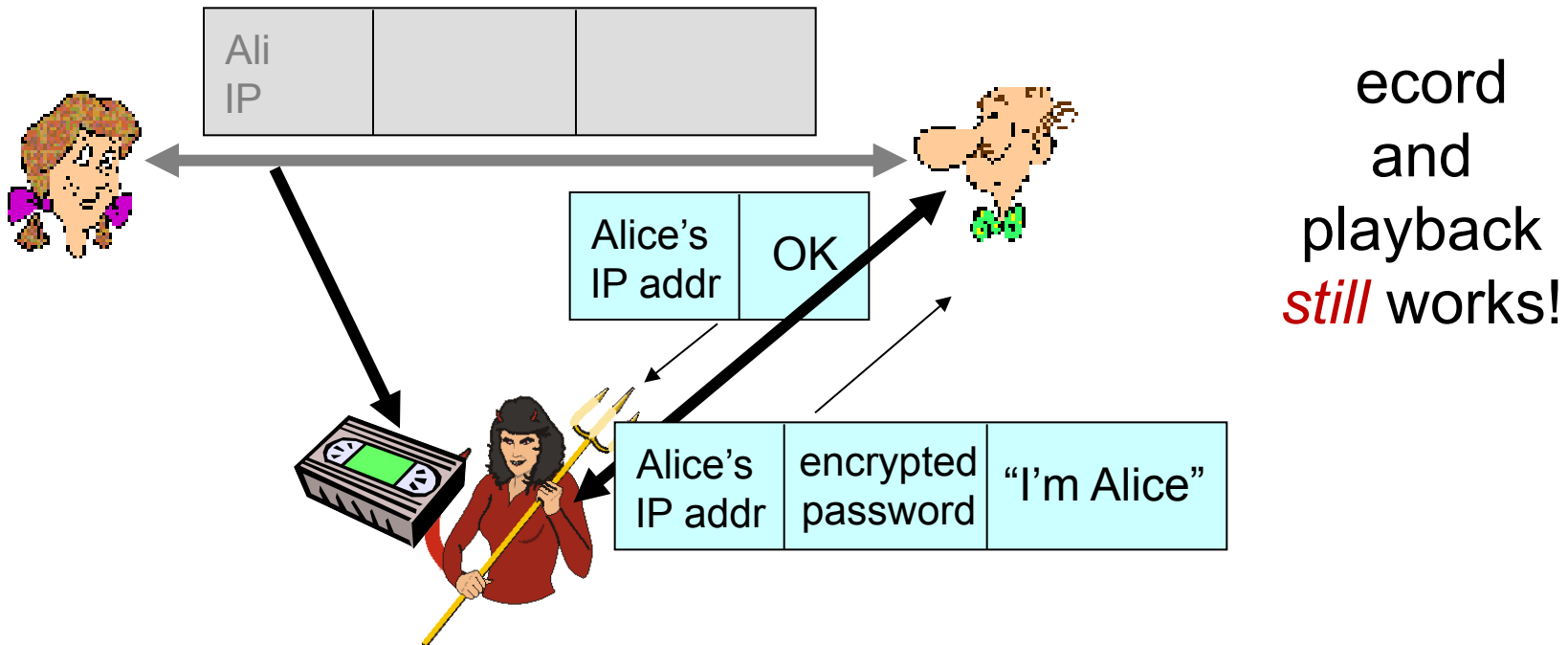
Authentication: yet ~~another try~~

ol ap3.1: Alice says "I am Alice" and sends her *encrypted* secret password to "prove" it.



Authentication: yet ~~another try~~

ol ap3.1: Alice says "I am Alice" and sends her *encrypted* secret password to "prove" it.

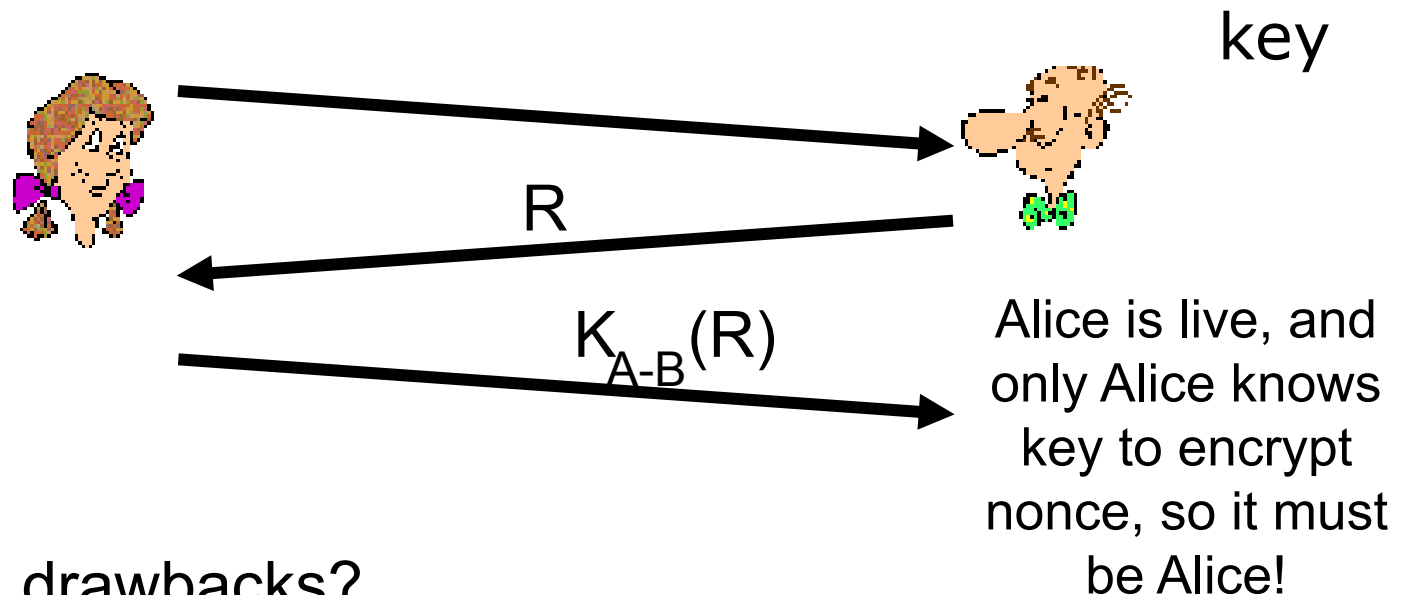


Authentication: yet ~~another try~~

Goal: avoid playback attack

Requirement: number (R) used only *once-in-a-lifetime*

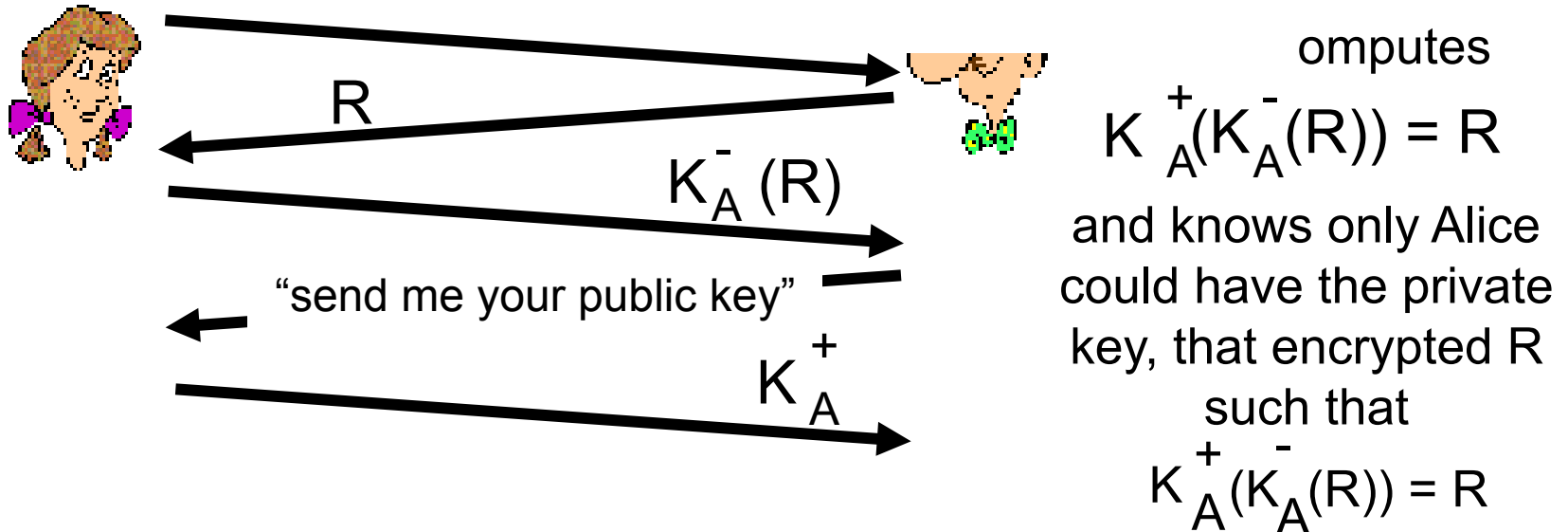
ap4.0: to prove Alice "live", Bob sends Alice



Authentication: ap5.0

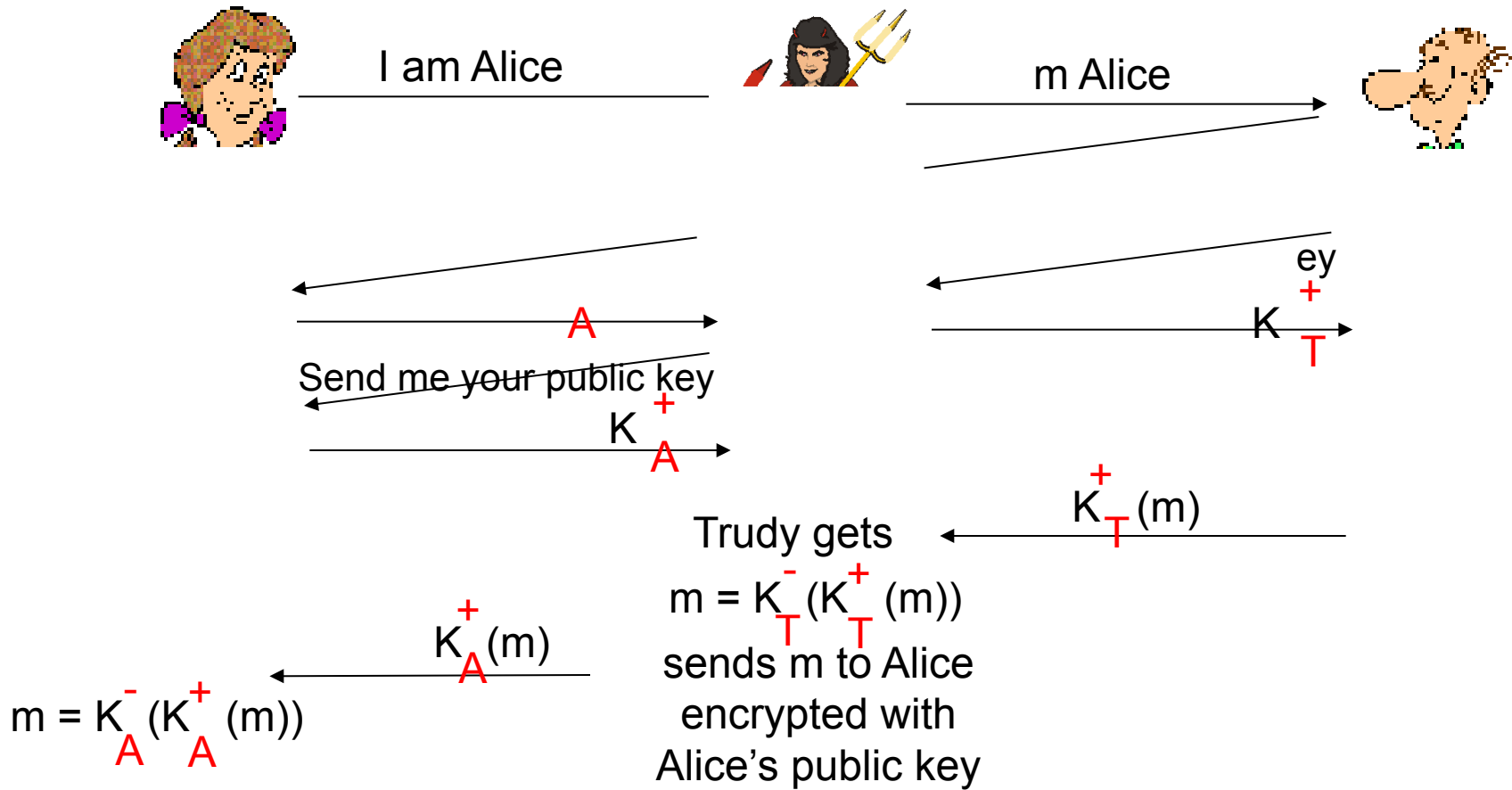
ap4.0 requires shared symmetric key

- ❖ can we authenticate using public key techniques?



~~hole~~

man (or woman) in the middle attack: Trudy poses as Alice (to Bob) and as Bob (to Alice)



ap5.0: security

~~hole~~

man (or woman) in the middle attack: Trudy poses as Alice (to Bob) and as Bob (to Alice)



difficu

- ❖ Bob receives everything that Alice sends, and vice versa. (e.g., so Bob, Alice can meet one week later and recall conversation!)
- ❖ problem is that Trudy receives all messages as well!

Chapter 8 roadmap

8.1 What is network security?

8.2 Principles of cryptography

8.5 Securing TCP connections: SSL

8.6 Network layer security: IPsec

8.7 Securing wireless LANs

8.8 Operational security: firewalls and
IDS

Digital ~~signatures~~

cryptographic technique analogous to
hand-written signatures:

- ❖ *verify* (Alice)
can prove to someone that Bob, and no
one else (including Alice), must have
signed document

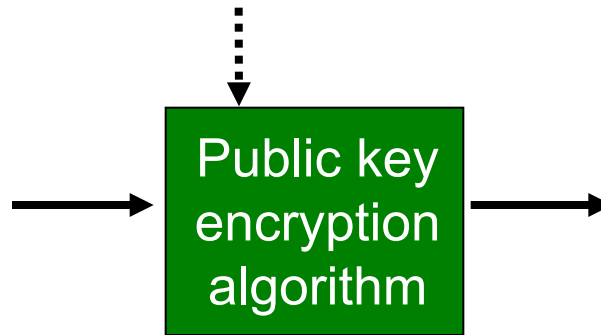
Digital ~~signatures~~

simple digital signature for message m :

- ❖ Bob signs m by encrypting with his private key K_B , creating message, $K_B(m)$

Bob's m

Dear Alice
Oh, how I have missed
you. I think of you all the
time! ... (blah blah blah)
Bob



$K_B(m)$

Bob's message,
 m , signed
(encrypted) with
his private key

Digital ~~signatures~~

- ❖ suppose Alice receives msg m , with signature:
 $m, K_B(m)$
- ❖ Alice verifies m signed by Bob by applying Bob's
public key K checks $K(K(m)) =$

- ❖ If K_B used Bob's private key. ust have
Alice thus verifies that:
 - ✓ Bob signed m

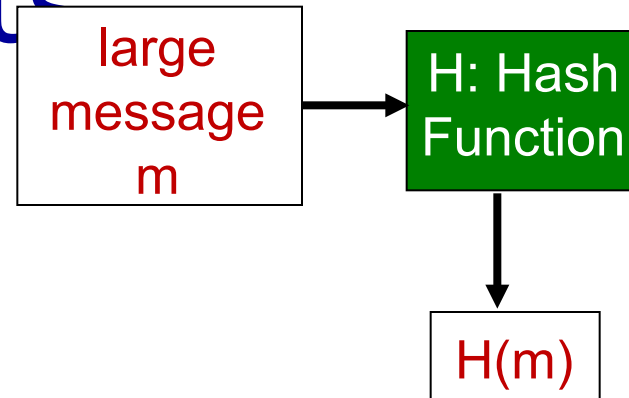
- ✓ no one else signed m

- ✓ Bob signed m and not m'

non-repudiation:

- ✓ Alice can take m , and signature $K_B(m)$ to
court and prove that Bob signed m

Message digests



computationally
expensive to public-
key-encrypt lo

goal: f

- easy- to-compute digital “fingerprint”
- ❖ apply hash function H to m , get fixed size message digest, $H(m)$.
- ❖ many-to-1
- ❖ produces fixed-size msg digest (fingerprint)
- ❖ given message digest x , computationally infeasible to find m such that $x = H(m)$

Internet checksum: poor crypto hash function

Internet checksum has some properties of hash function:

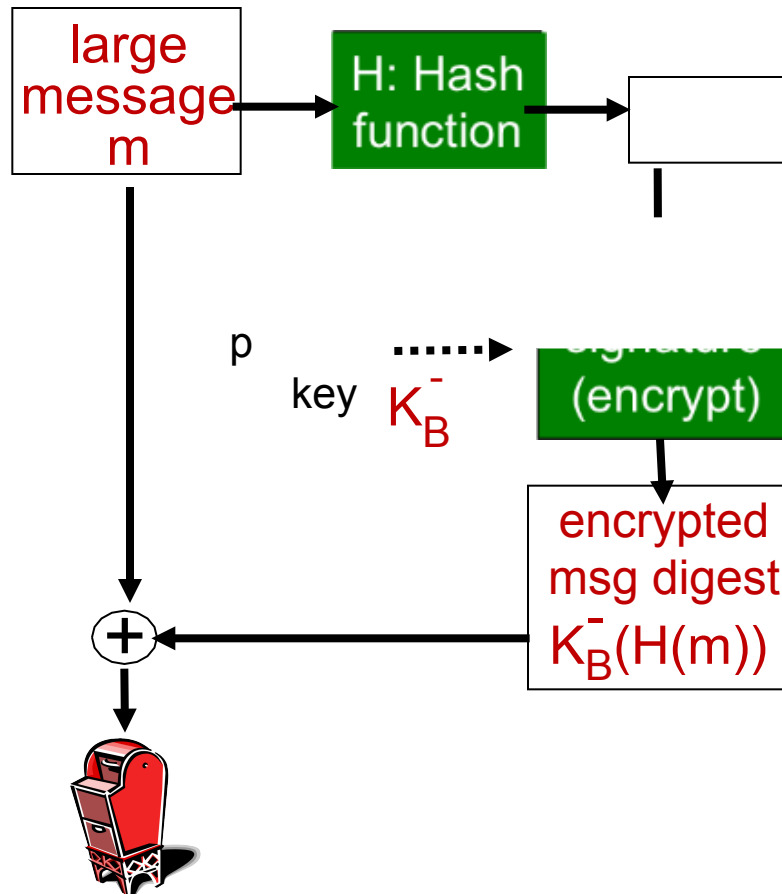
- ✓ produces fixed length digest (16-bit sum) of message

B
✓

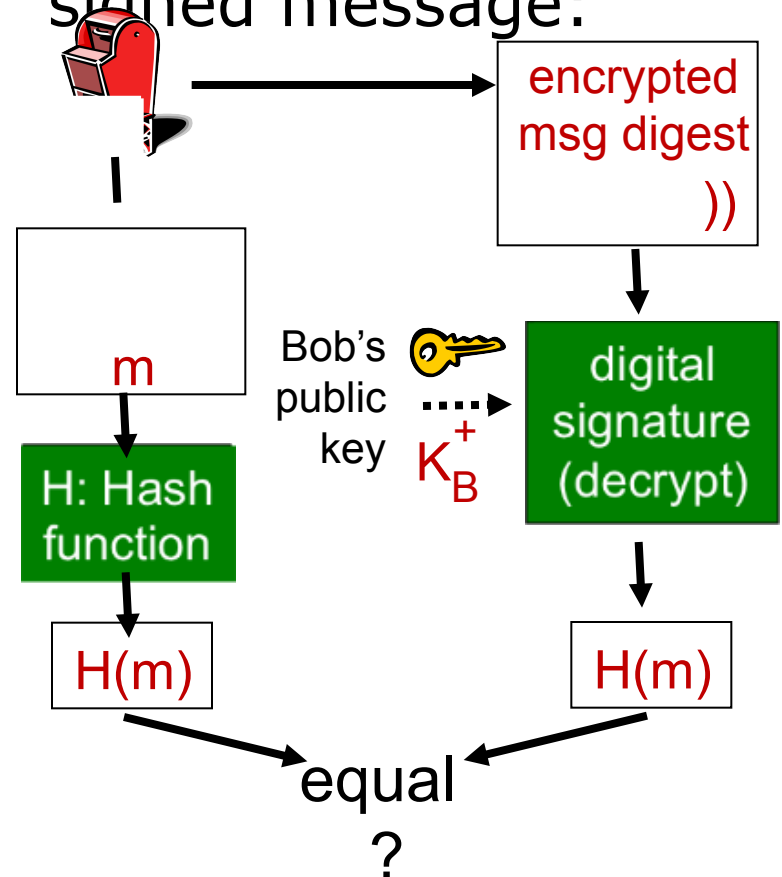
<u>message</u>	<u>ASCII format</u>		<u>message</u>	<u>ASCII format</u>
I O U 1	49 4F 55 31		I O U <u>9</u>	49 4F 55 <u>39</u>
0 0 . 9	30 30 2E 39		0 0 . <u>1</u>	30 30 2E <u>31</u>
9 B O B	39 42 D2 42		9 B O B	39 42 D2 42
	<u>B2 C1 D2 AC</u>	different messages		<u>B2 C1 D2 AC</u>
		but identical checksums!		

Digital signature = signed message digest

Bob sends digitally signed message:



Alice verifies signature, integrity of digitally signed message:



Hash function algorithms

- ❖ MD5 hash function widely used (RFC 1321)

- computes 128-bit message digest in 4-step

CO
X

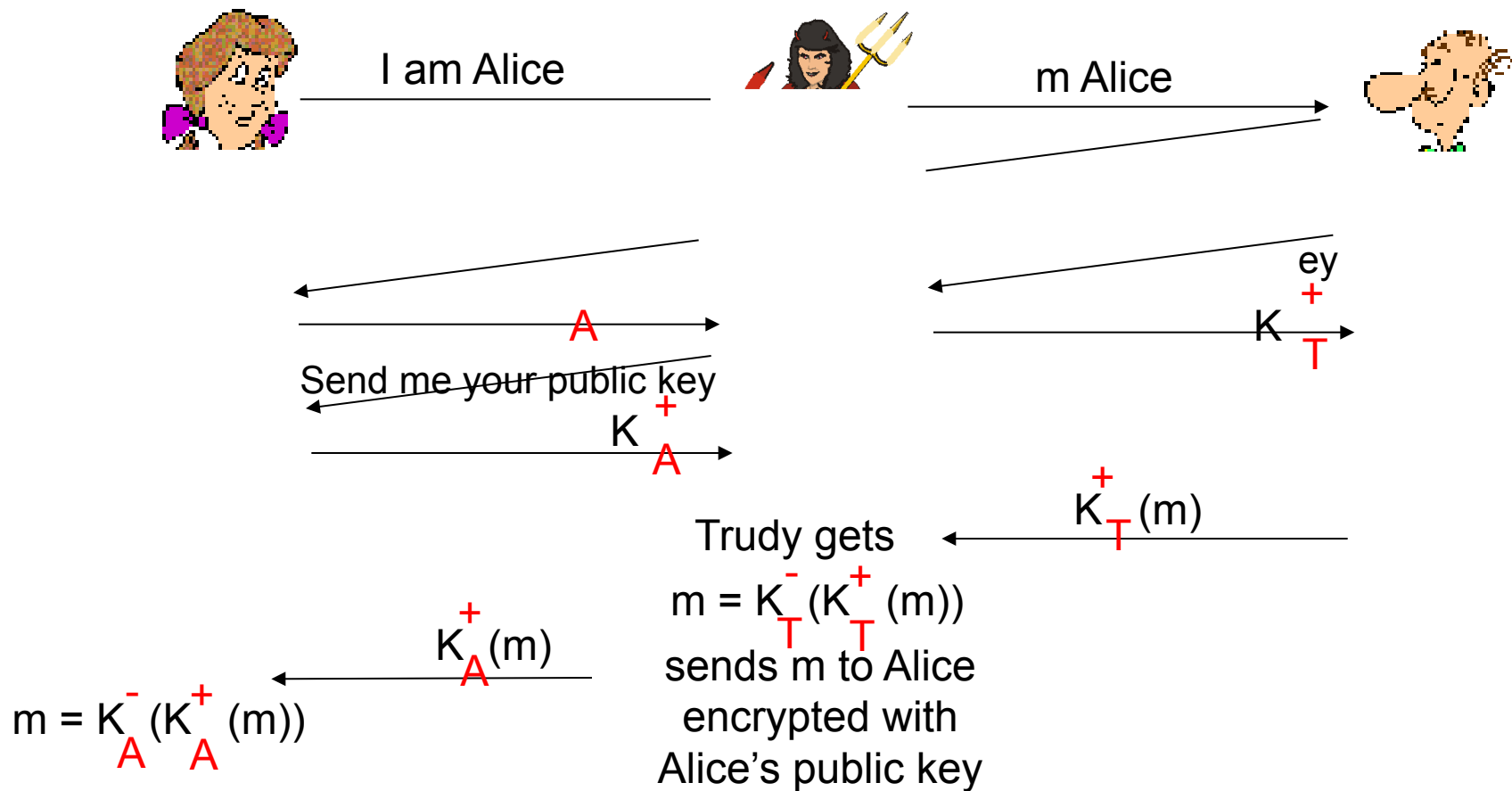
qual to

- ❖ SHA-1 is also used

- US standard [NIST, FIPS PUB 180-1]
- 160-bit message digest

Recall: ap5.0 security hole

man (or woman) in the middle attack: Trudy poses as Alice (to Bob) and as Bob (to Alice)



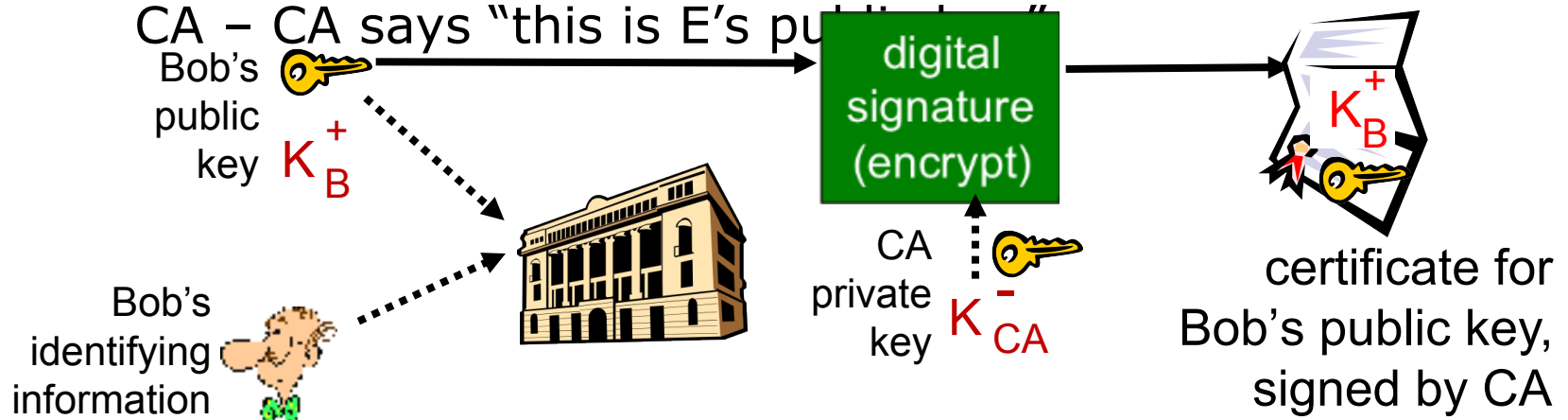
Public-key certification

- ❖ motivation: Trudy plays pizza prank on Bob
 - Trudy create
 - Tru
 - Trudy sends order to Pizza Store
 - Trudy sends to Pizza Store her public key, but says it's Bob's public key
 - Pizza Store verifies signature; then delivers four pepperoni pizzas to Bob
 - Bob doesn't even like pepperoni

Certification authorities

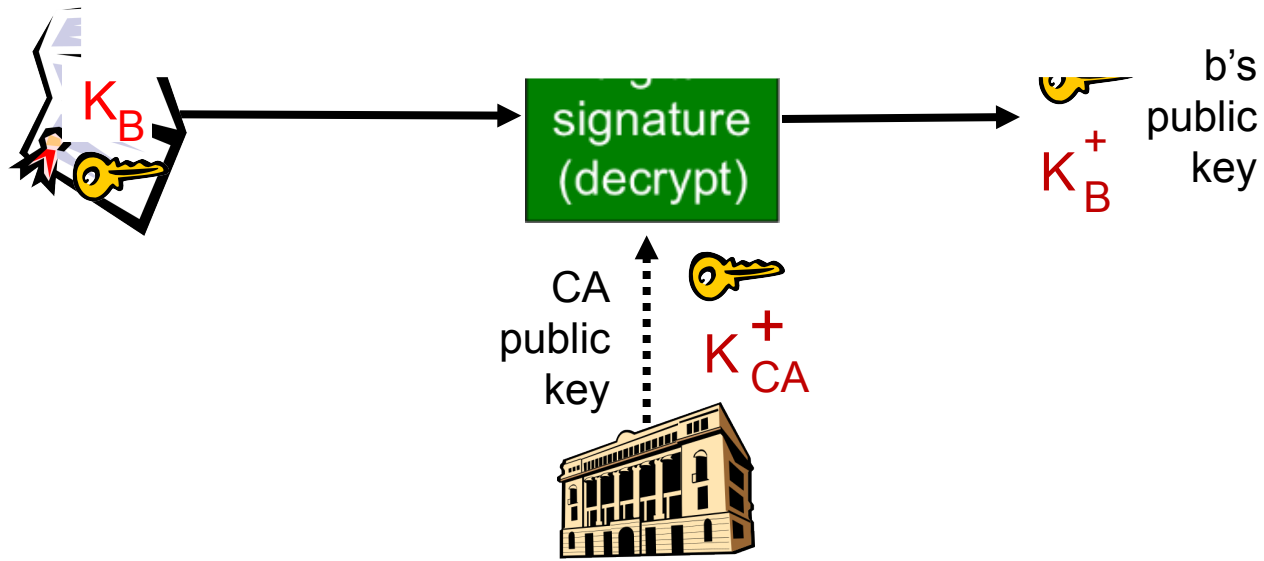
- ❖ *certification authority (CA)*: binds public key to particular entity, E.
- ❖ E (person, rout public key with

- CA
- certificate containing E's public key digitally signed by CA – CA says "this is E's public key"



Certification ~~authorities~~

- ❖ when Alice wants Bob's public key:
 - gets Bob's certificate (Bob or elsewhere).
 - apply CA's public key to Bob's certificate, get Bob's pu



Chapter 8 roadmap

8.1 What is network security?

8.2 Principles of cryptography

8.5 Securing TCP connections: SSL

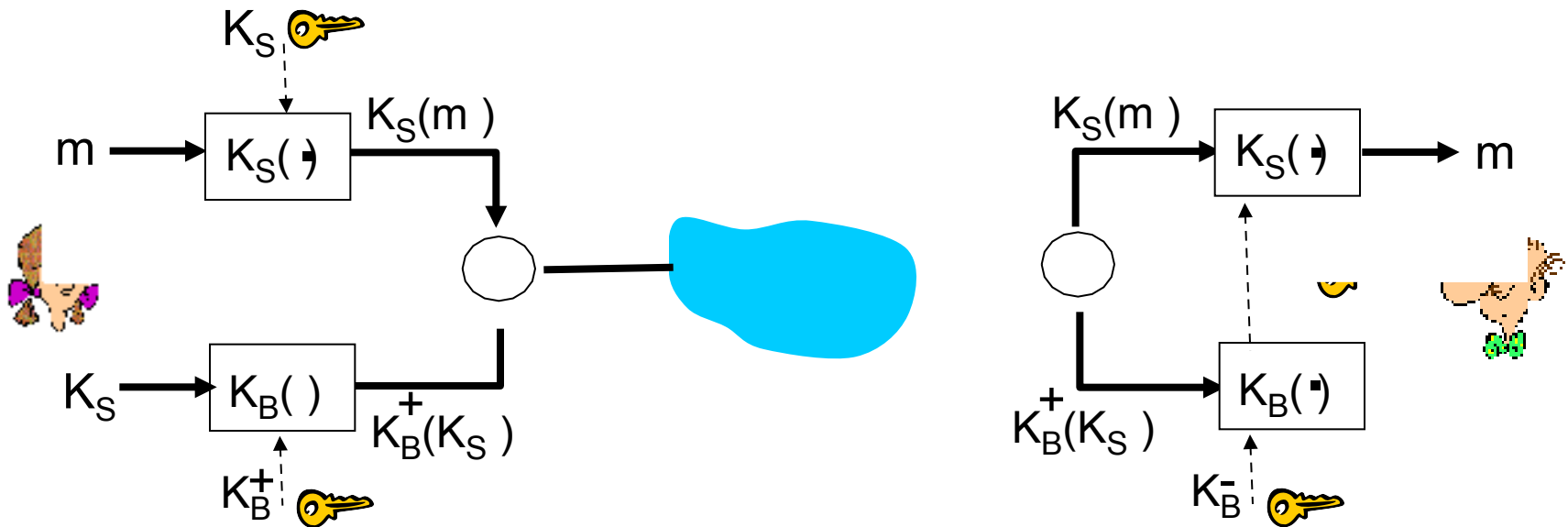
8.6 Network layer security: IPsec

8.7 Securing wireless LANs

8.8 Operational security: firewalls and
IDS

Secure e-mail

- ❖ Alice wants to send confidential e-mail, m , to Bob.

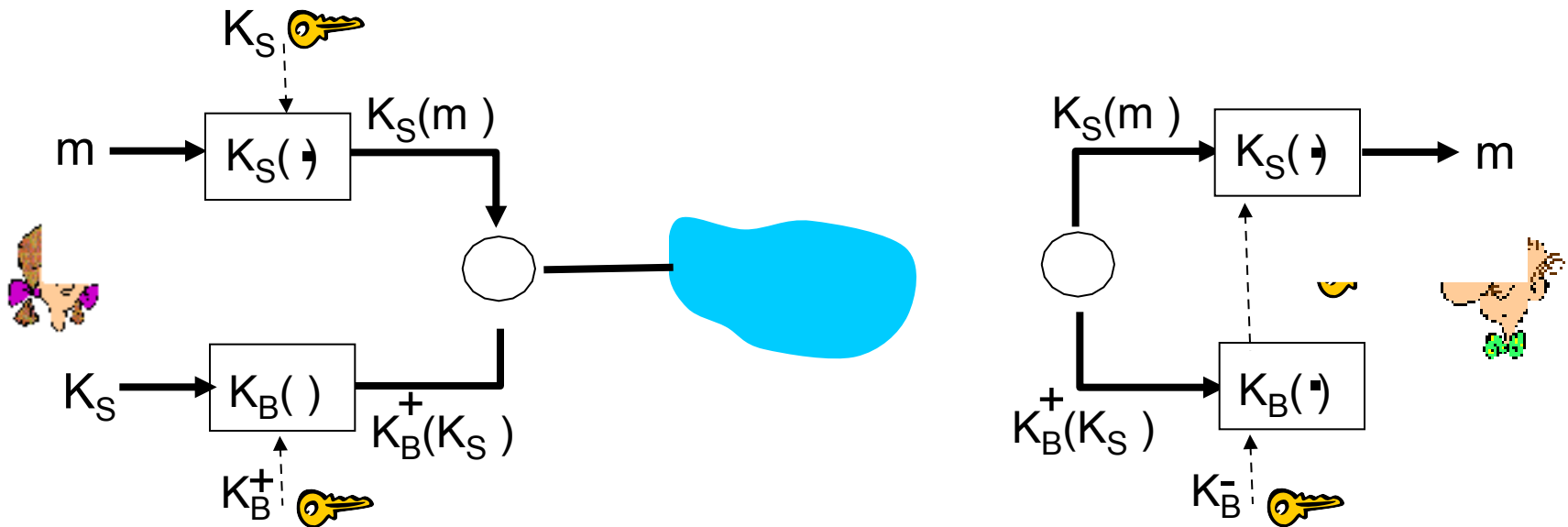


Alice:

- ❖ generates random *symmetric* private key, K_S
- ❖ encrypts message with K_S (for efficiency)
- ❖ also encrypts K_S with Bob's public key
- ❖ sends both $K_S(m)$ and $K_B(K_S)$ to Bob

Secure e-mail

- ❖ Alice wants to send confidential e-mail, m , to Bob.

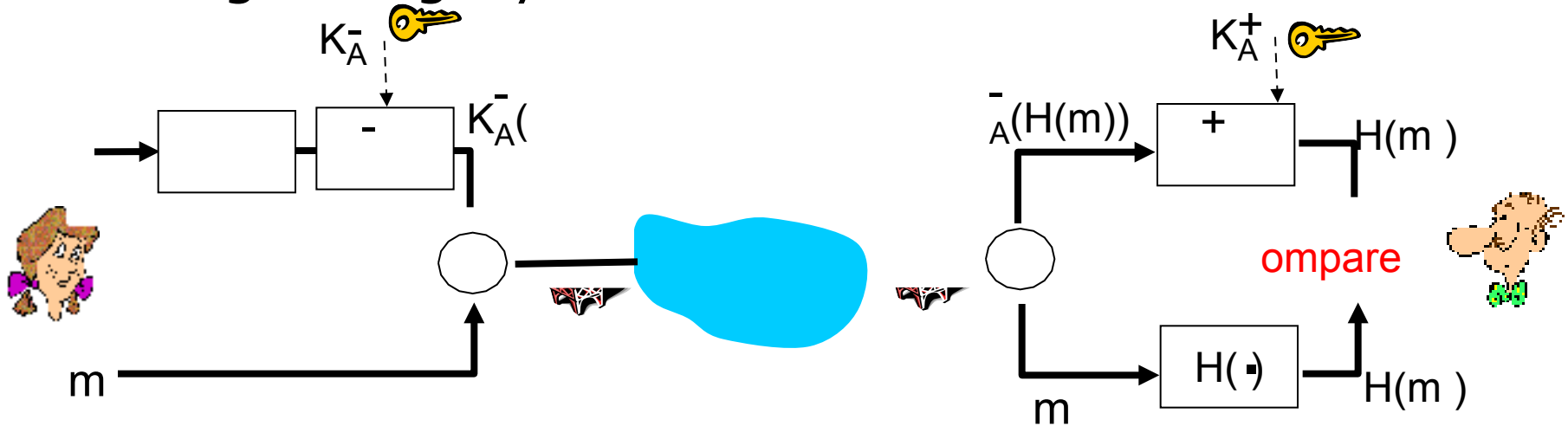


Bob:

- ❖ uses his private key to decrypt and recover K_S
- ❖ uses K_S to decrypt $K_S(m)$ to recover m

Secure e-mail (continued)

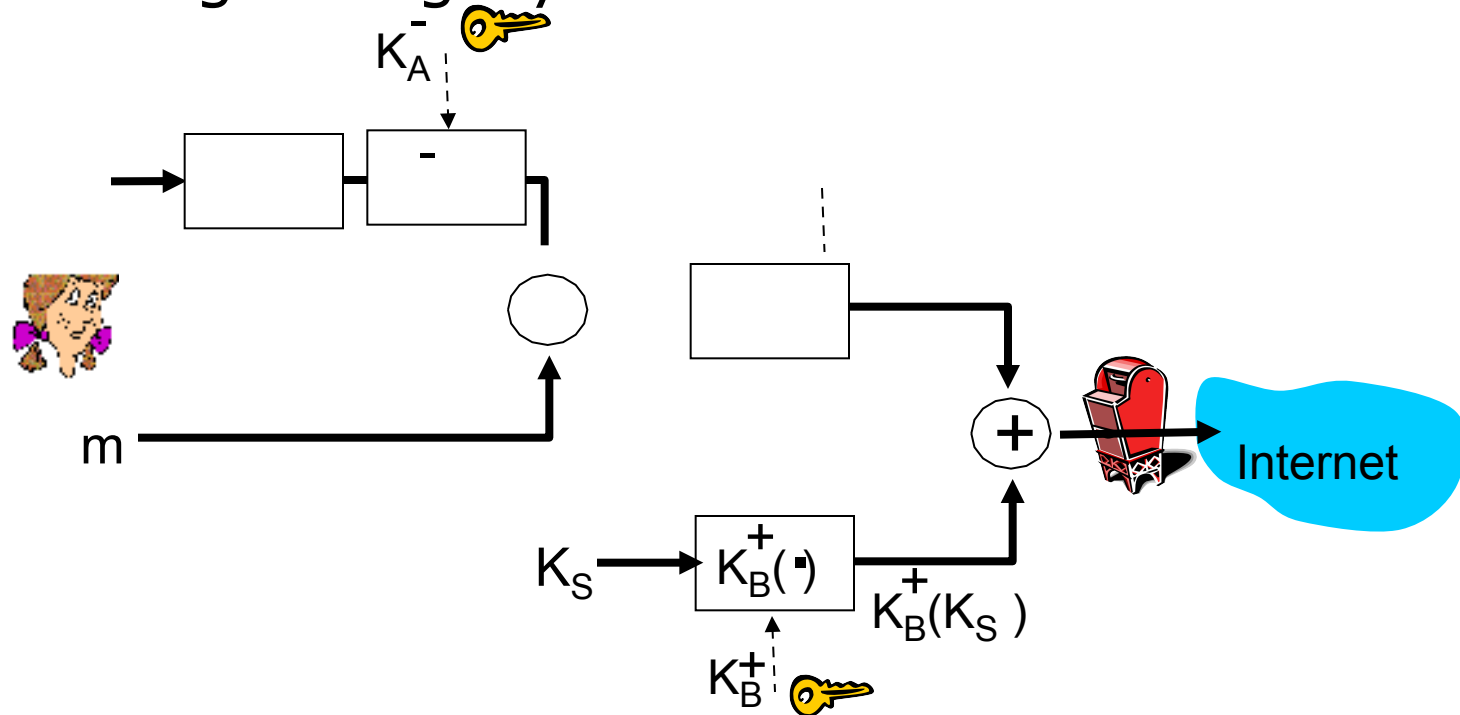
- ❖ Alice wants to provide sender authentication
message integrity



- ❖ Alice digitally signs message
- ❖ sends both message (in the clear) and digital signature

Secure e-mail (continued)

- ❖ Alice wants to provide secrecy, sender authentication, message integrity.



Alice uses three keys: her private key, Bob's public key, newly created symmetric key

Chapter 8 roadmap

8.1 What is network security?

8.2 Principles of cryptography

8.5 Securing TCP connections: SSL

8.6 Network layer security: IPsec

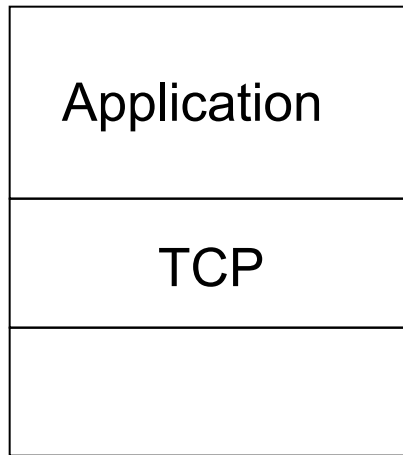
8.7 Securing wireless LANs

8.8 Operational security: firewalls and
IDS

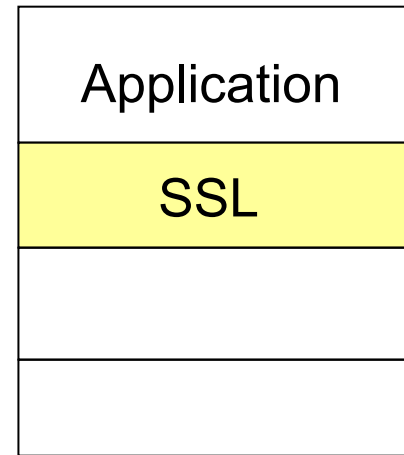
SSL: Secure Sockets Layer

- ❖ widely deployed security protocol
 - supported by almost all browsers, web servers
- ❖ original goals:
 - Web e-commerce transactions
 - confidentiality (especially for sensitive information)
- ❖ mechanism defined in [RFC 2246, 1994], implementation: Netscape
- ❖ variation -TLS: transport layer security, RFC 2246
- ❖ provides
 - *confidentiality*
 - *integrity*
- ❖ optional client authentication
- minimum hassle in doing business with new merchant
- ❖ available to all TCP applications
 - secure socket interface

SSL and TCP/IP



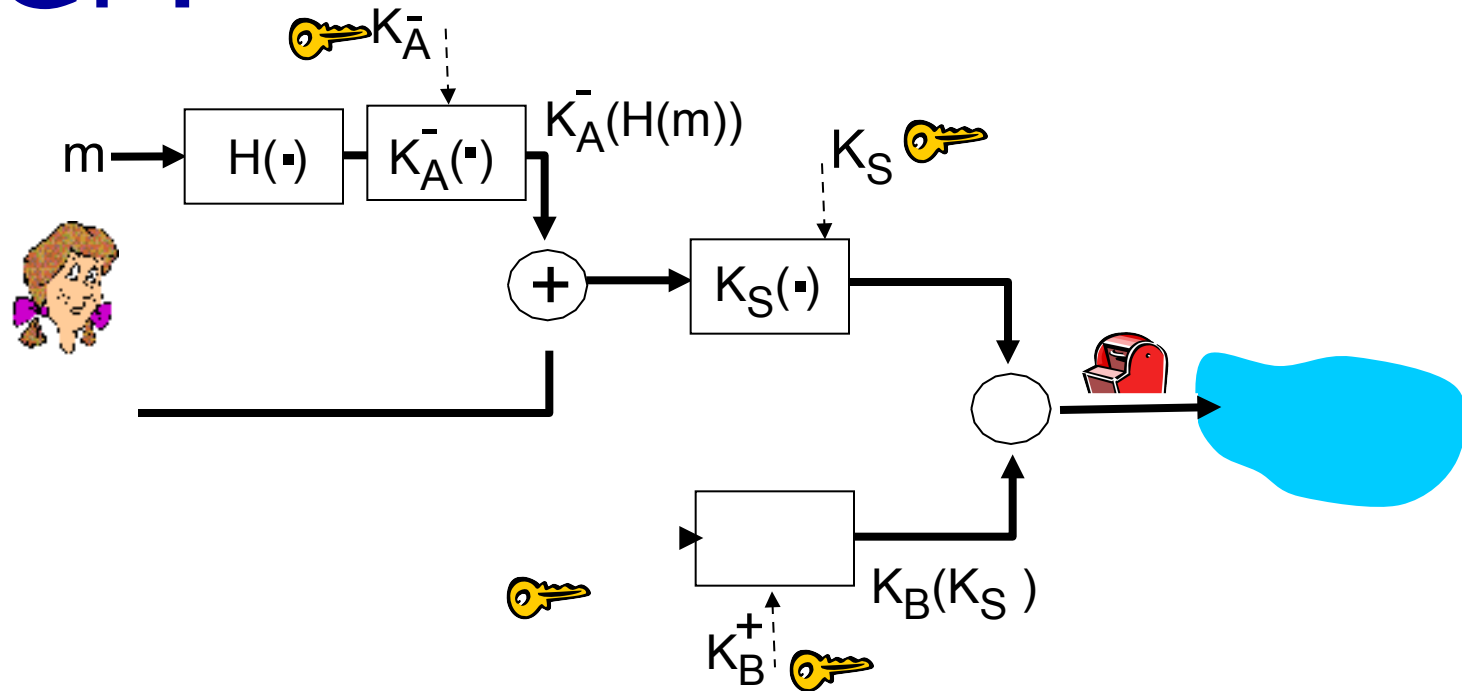
normal application



application with SSL

- ❖ SSL provides application programming interface (API) to applications
- ❖ C and Java SSL libraries/classes readily available

Could do something like ~~PGP:~~

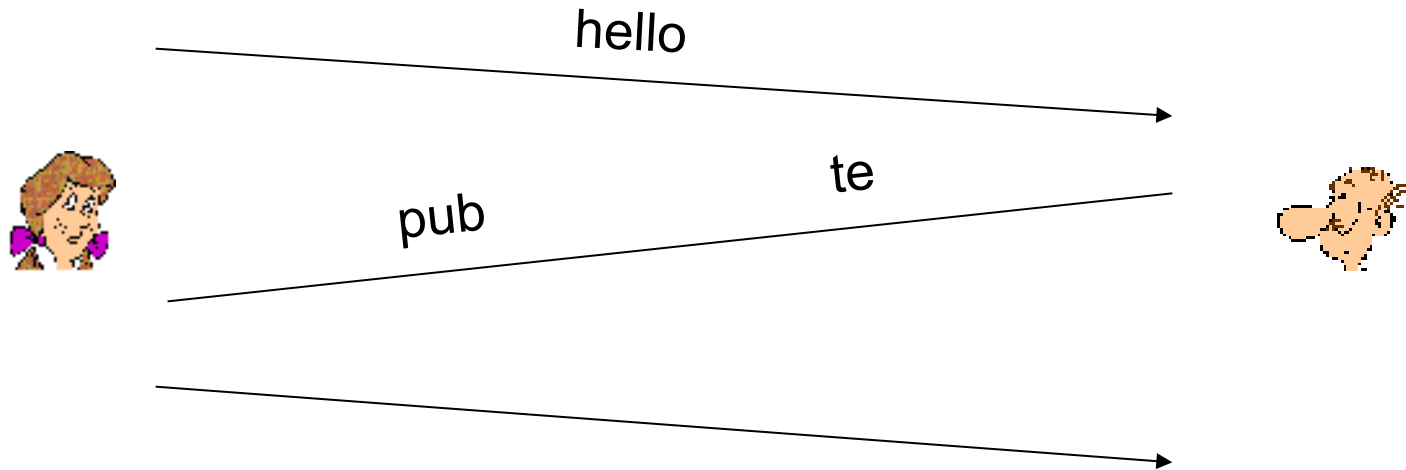


- ❖ but want to send byte streams & interactive data
- ❖ want set of secret keys for entire connection
- ❖ want certificate exchange as part of protocol:
handshake phase

Toy SSL: a simple secure channel

- ❖ *handshake*: Alice and Bob use their certificates, private keys to authenticate each other and share a shared secret
- shar
- ❖ *data transfer*: data to be transferred is broken up into series of records
- ❖ *connection closure*: special messages to securely close connection

Toy: a simple handshake



MS: master secret

EMS: encrypted master secret

Toy: key derivation

❖ considered bad to use same key for more than one cryptographic operation

- use different keys for authentication code (MAC) and encryption

- K_c
client to server

- M_c = MAC key for data sent from client to server

- K_s = encryption key for data sent from server to client

- M_s = MAC key for data sent from server to client

Toy: data records

- ❖ why not encrypt data in constant stream as we write it to TCP?
 - where would we put the MAC? If at end, no message integrity until all data processed.
 - e.g., with insta can we do
- eac
- receiver can act on each record as it arrives
- ❖ issue: in record, receiver needs to distinguish MAC from data
 - want to use variable-length records



Toy: sequence numbers

❖ *problem:* attacker can capture and replay record or re-order records



- MA $\times$
- note: no sequence number field

❖ *problem:* attacker could replay all records

❖ *solution:* use nonce

Toy: control information

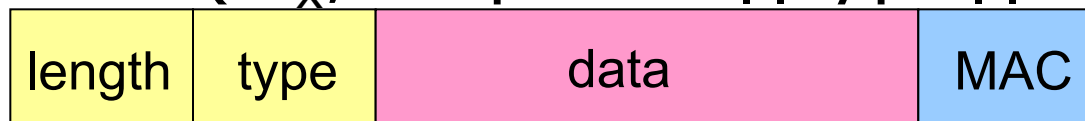
❖ *problem:* truncation attack:

- attacker forges TCP connection close segment

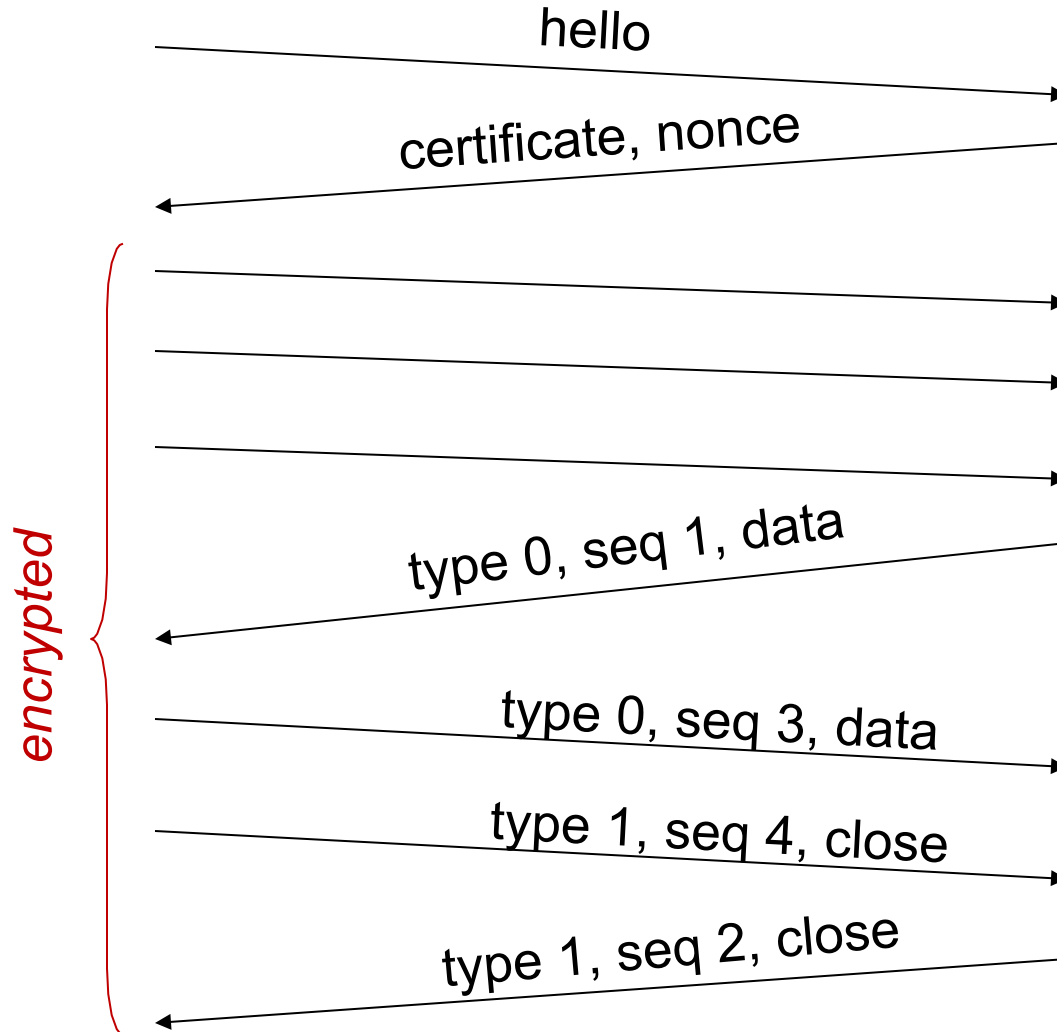
❖ *solution:* record types, with one type for closure

- type 0 for data; type 1 for closure

❖ $MAC = MAC(M_x, \text{sequence} || \text{type} || \text{data})$



Toy SSL: summary



Toy SSL isn't complete

- ❖ how long are fields?
- ❖ which encryption protocols?

enc

- allow client and server to choose together specific algorithm before data transfer

SSL cipher suite

❖ cipher suite

- public-key algorithm
- symmetric encryption algorithm



cipher suites

❖ negotiation: client, server agree on cipher suite

- client offers choice
- server picks one

common SSL symmetric ciphers

- DES – Data Encryption
standard: block

ck

cipher 2: block

- RC4 – Rivest Cipher 4:
stream

SSL Public key encryption

- RSA

Real SSL: handshake (1)

Purpose

1. server authentication
2. key exchange
3. establish session ID
4. client authentication (optional)

Real SSL: handshake (2)

1. client sends list of algorithms it supports, along with client nonce
2. server choose om list; sends
3. publi ret,
encrypts with server's public key, sends to server
4. client and server independently compute encryption and MAC keys from pre_master_secret and nonces
5. client sends a MAC of all the handshake messages

Real SSL: handshaking (3)

last 2 steps protect handshake from tampering



- ❖ man-in-the-middle attacks
- ❖ algorithms from list
- ❖ last 2 steps prevent this
 - last two messages are encrypted

nger

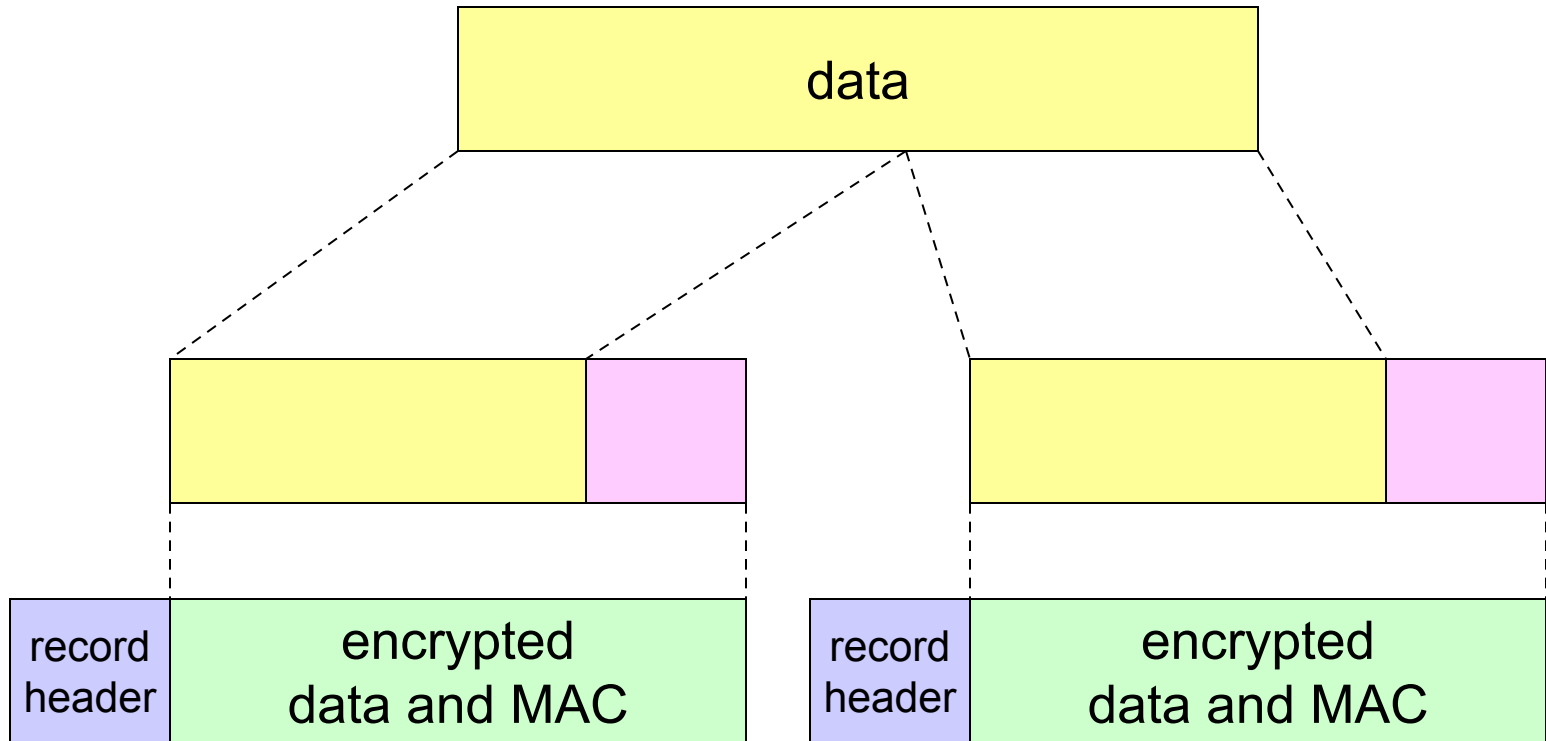
Real SSL: handshaking (4)

- ❖ why two random nonces?
- ❖ suppose Trudy sniffs all messages between Alic

with ce
of records

- Bob (Amazon) thinks Alice made two separate orders for the same thing
- solution: Bob sends different random nonce for each connection. This causes encryption keys to be different on the two days
- Trudy's messages will fail Bob's integrity check

SSL record protocol

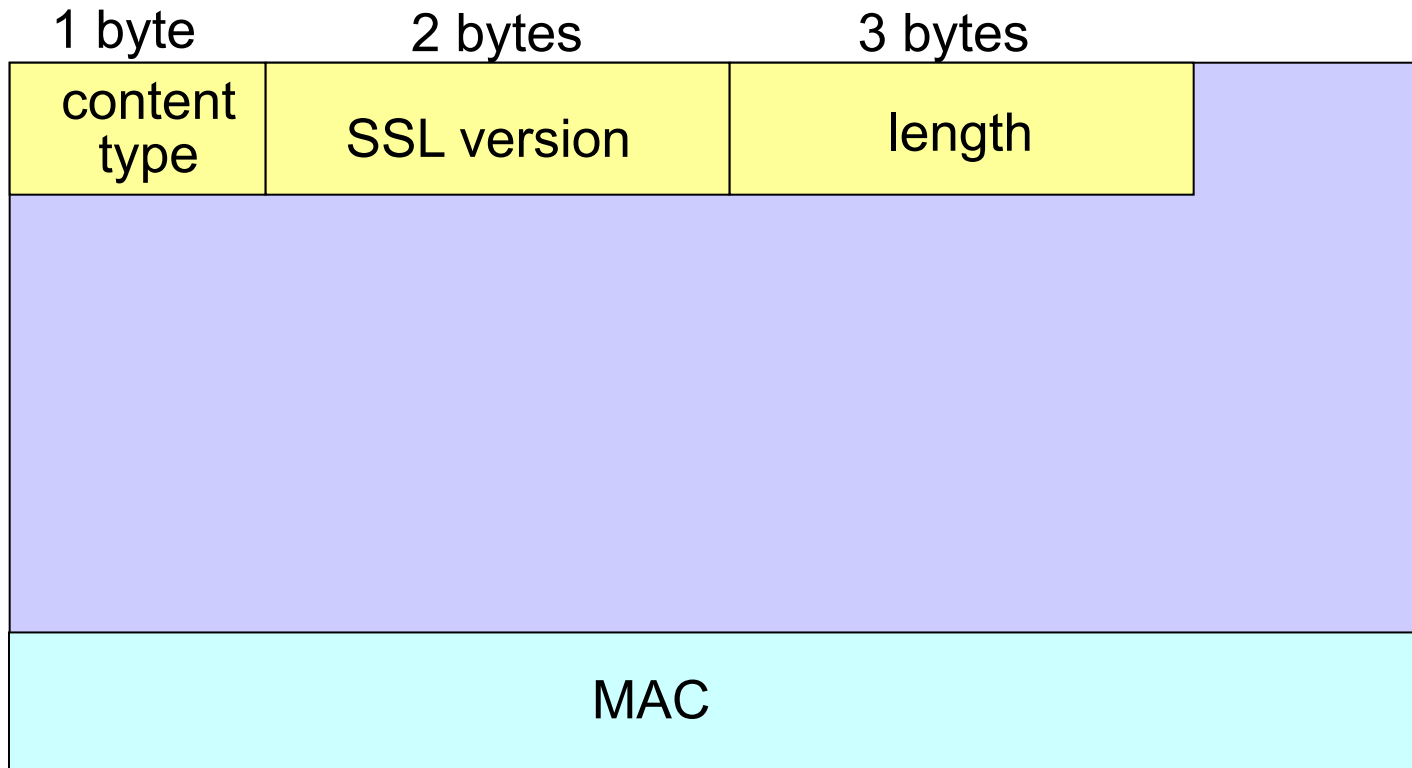


record header: content type; version; length

MAC: includes sequence number, MAC key M_x

fragment: each SSL fragment 2^{14} bytes (~ 16 Kbytes)

SSL record format

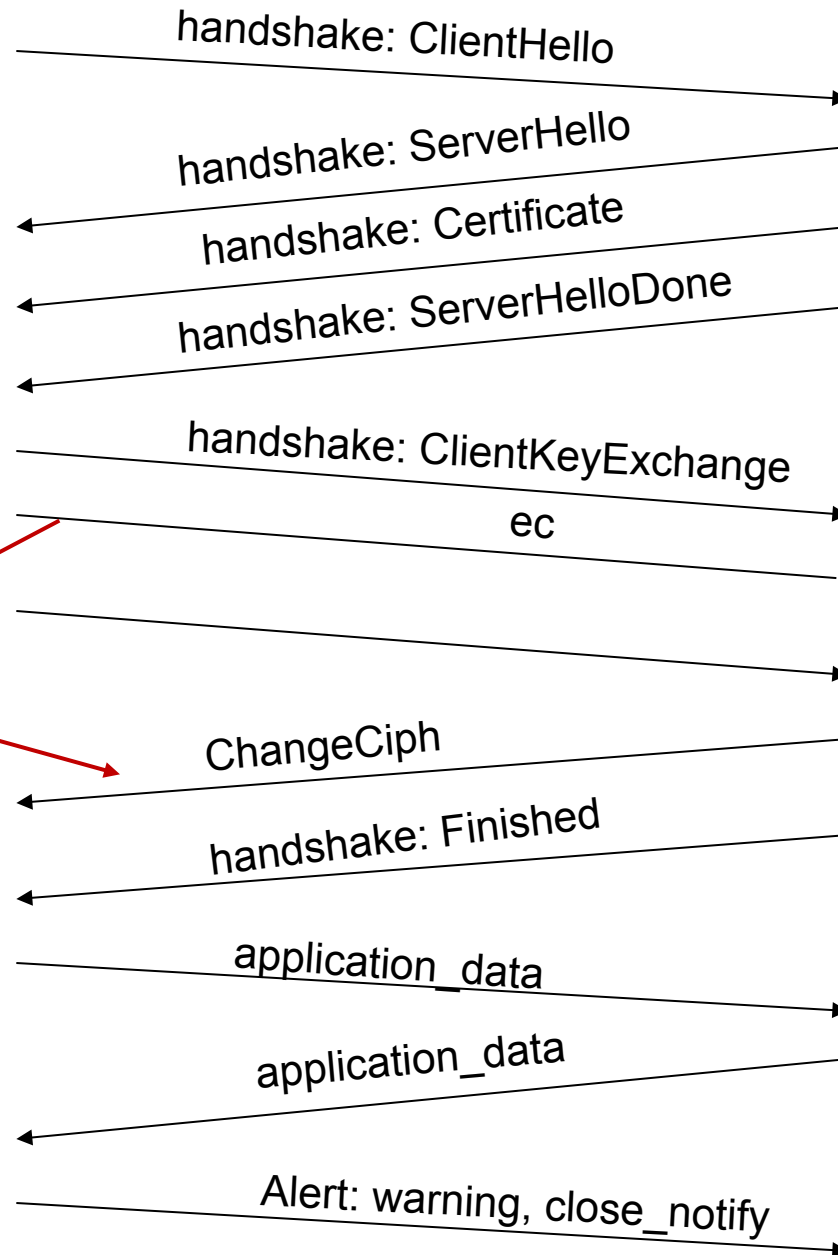


data and MAC encrypted (symmetric algorithm)

Real SSL connection

is encr

TCP FIN follows



Key derivation

- ❖ client nonce, server nonce, and pre-master secret input into pseudo random-number generator.
 - produces maste

block"

- because of resumption: TBD
- ❖ key block sliced and diced:
 - client MAC key
 - server MAC key
 - client encryption key
 - server encryption key
 - client initialization vector (IV)
 - server initialization vector (IV)

Chapter 8 roadmap

8.1 What is network security?

8.2 Principles of cryptography

8.5 Securing TCP connections: SSL

8.6 Network layer security: IPsec

8.7 Securing wireless LANs

8.8 Operational security: firewalls and
IDS

What is network-layer confidentiality ?

between two network entities:

- ❖ sending entity encrypts datagram payload, payl
- ❖ all data sent from one entity to other would be hidden:
 - web pages, e-mail, P2P file transfers, TCP SYN packets ...
- ❖ "blanket coverage"

Virtual Private Networks (VPNs)

motivation:

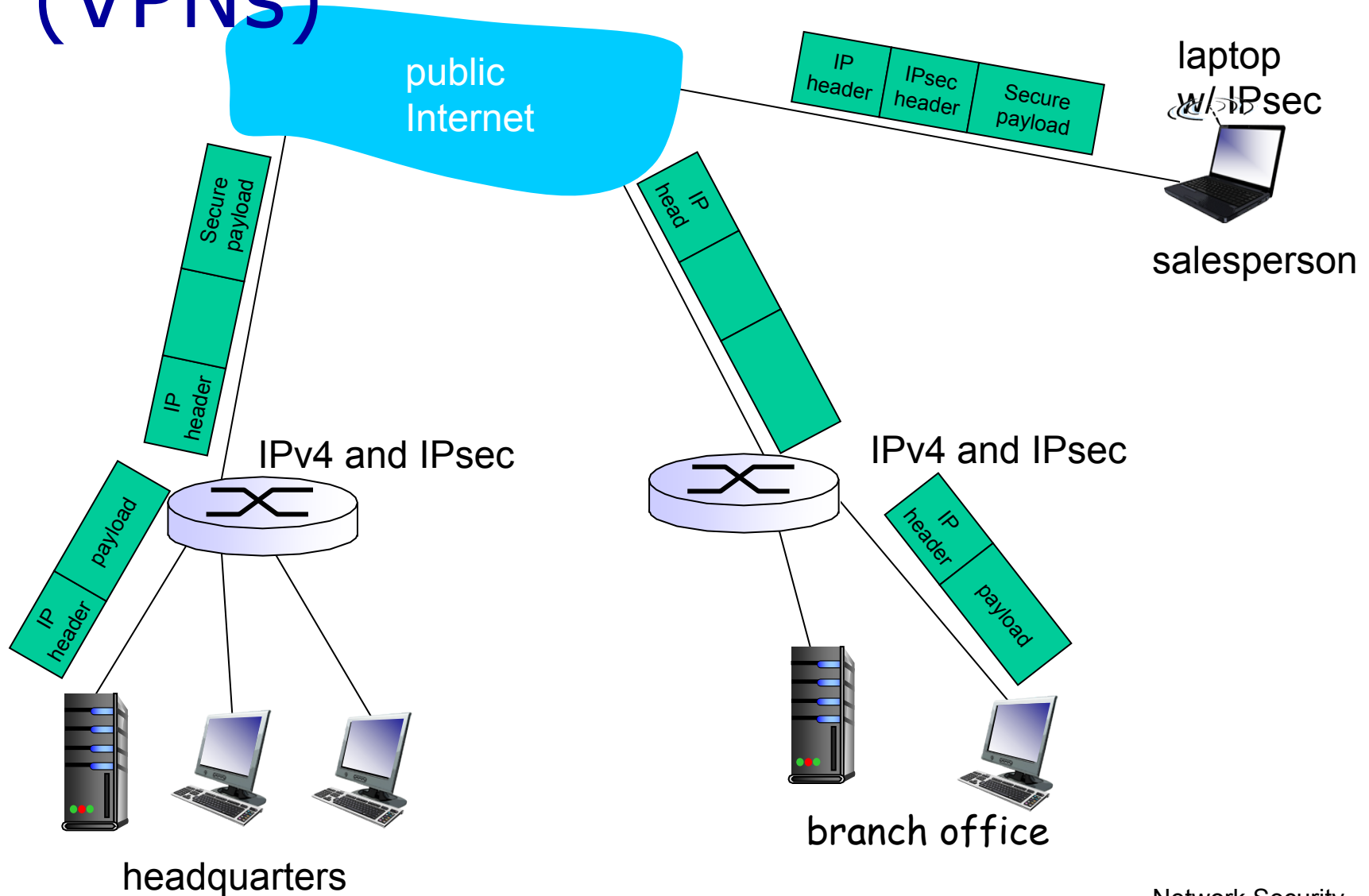
❖ institutions often want private networks for security.

- costly: separate infrastructure, DNS

❖ VPN: traffic is sent over public Internet instead

- encrypted before entering public Internet
- logically separate from other traffic

Virtual Private Networks (VPNs)

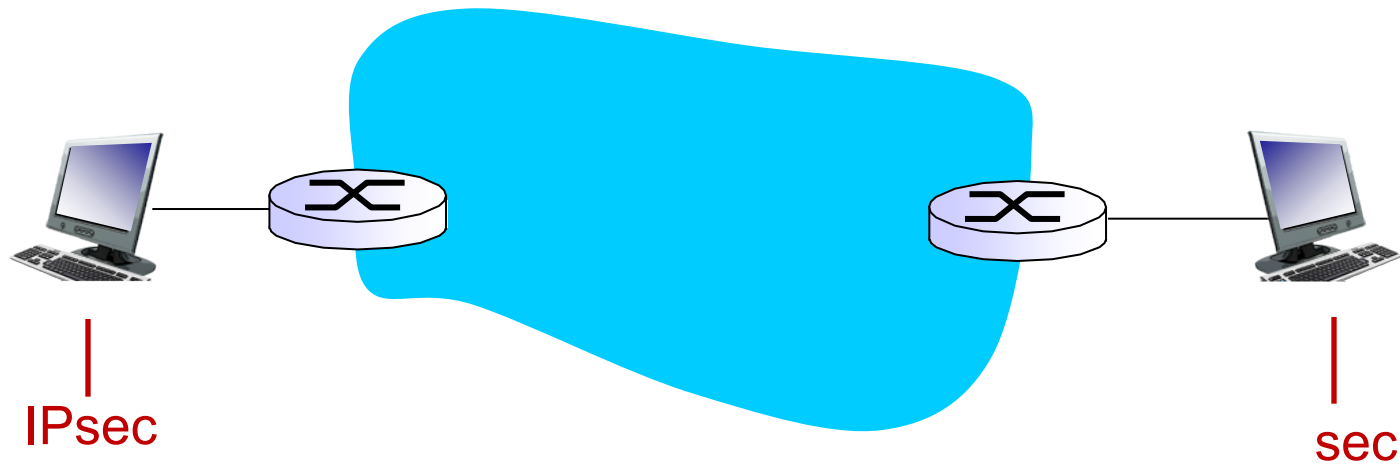


IPsec services

- ❖ data integrity
- ❖ origin authentication

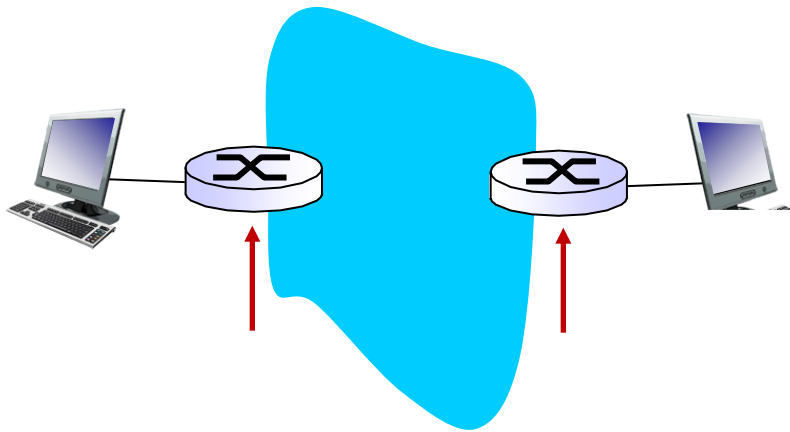
- ❖ two protocols providing different service models:
 - AH
 - ESP

IPsec transport mode

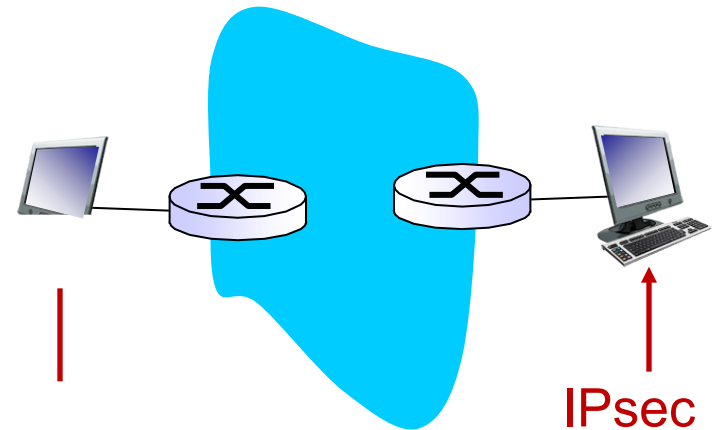


- ❖ IPsec datagram emitted and received by end-system
- ❖ protects upper level protocols

IPsec – tunneling mode



❖ edge routers IPsec-aware



❖ hosts IPsec-aware

Two IPsec protocols

- ❖ Authentication Header (AH) protocol
 - provides source authentication & data integrity but
 - provides confidentiality
 - integrity, *and confidentiality*
 - more widely used than AH

Four combinations are possible!

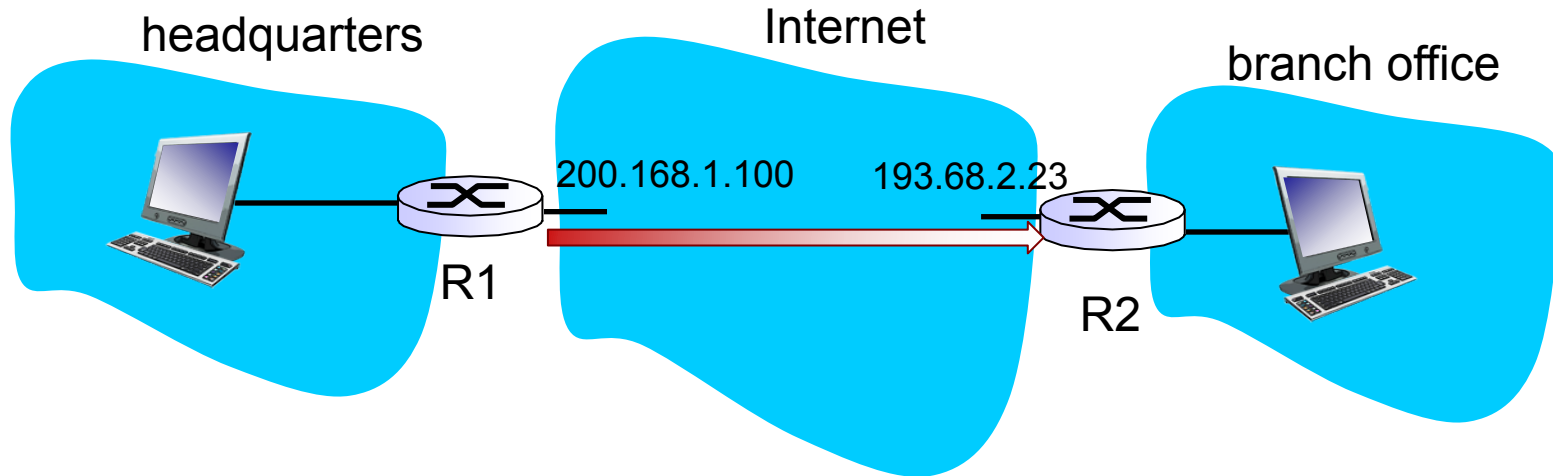
Host mode with A	Host mode SP
Tunnel mode with AH	Tunnel mode with ESP

most common and
most important

Security associations (SAs)

- ❖ before sending data, "security association (SA)" established from sending to receiving ent
- ❖ *endi* *state information* about SA
 - recall: TCP endpoints also maintain state info
 - IP is connectionless; IPsec is connection-oriented!
- ❖ how many SAs in VPN w/ headquarters, branch office, and n traveling salespeople?

Example SA from R1 to R2



R1 st

- ❖ 32-bit SA identifier: *Security Parameter Index (SPI)*
- ❖ origin SA interface (200.168.1.100)
- ❖ destination SA interface (193.68.2.23)
- ❖ type of encryption used (e.g., 3DES with CBC)
- ❖ encryption key
- ❖ type of integrity check used (e.g., HMAC with MD5)
- ❖ authentication key

Security Association Database (SAD)

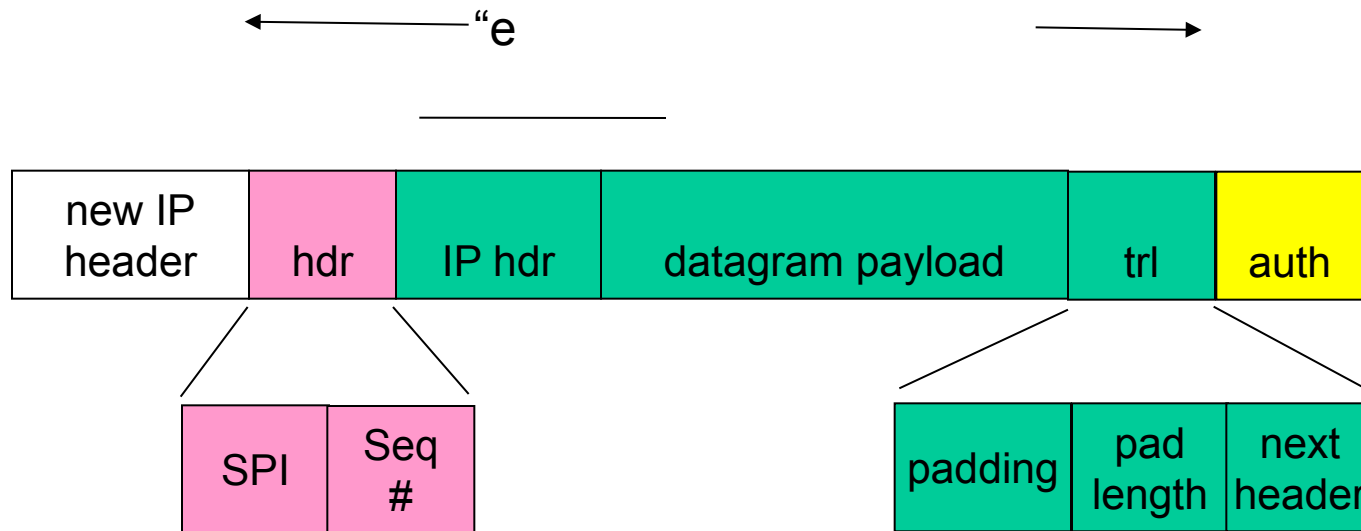
- ❖ endpoint holds SA state in *security association database (SAD)*, where it can locate the *essing*.

SAD

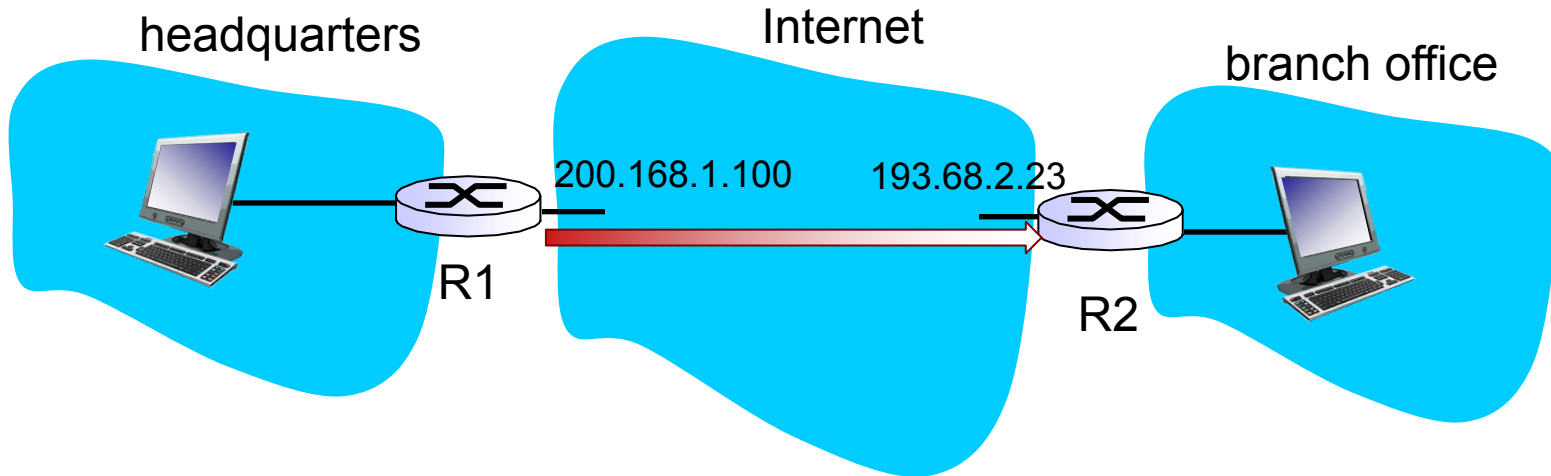
- ❖ when sending IPsec datagram, R1 accesses SAD to determine how to process datagram.
- ❖ when IPsec datagram arrives to R2, R2 examines SPI in IPsec datagram, indexes SAD with SPI, and processes datagram accordingly

IPsec datagram

focus for now on tunnel mode with ESP

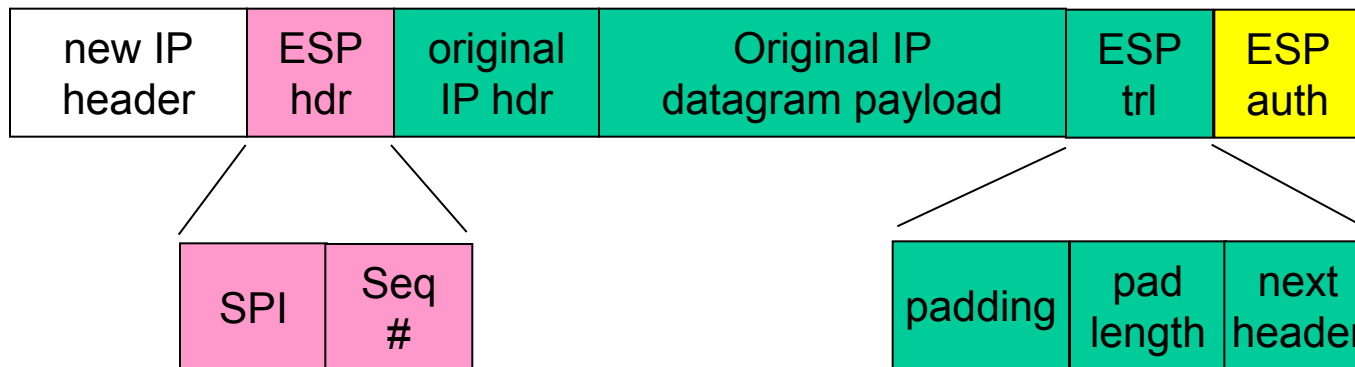


What happens?



← "enchilada" authenticated →

← encrypted →



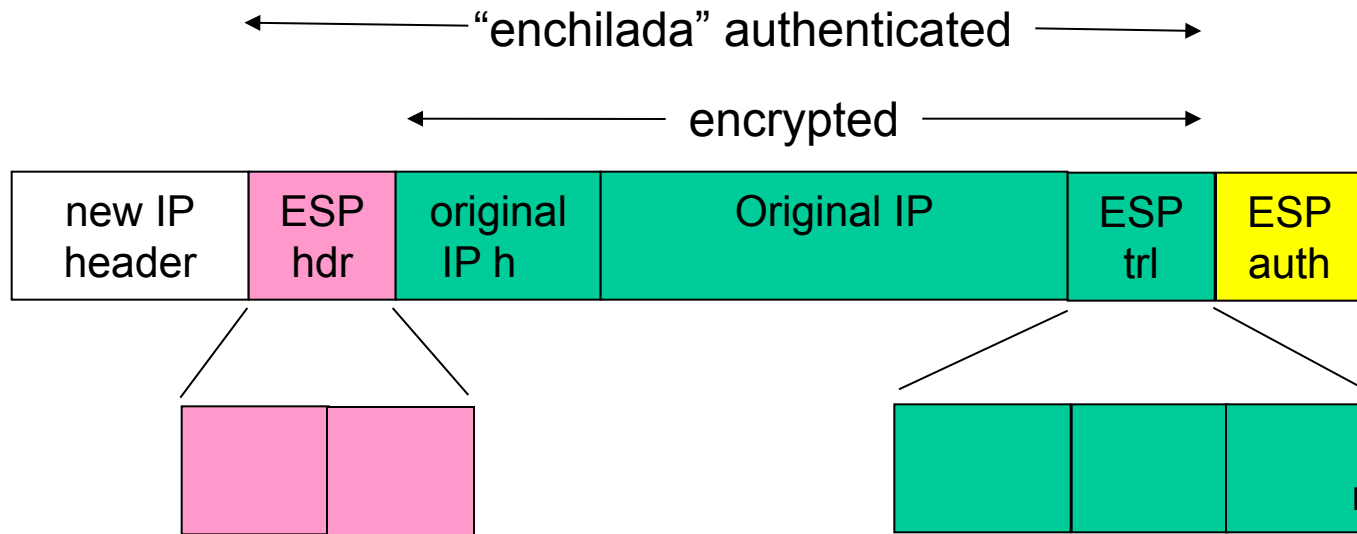
R1: convert original datagram to IPsec datagram

- ❖ appends to back of original datagram (which includes original header fields!) an “ESP trailer” field.

y

- ❖ appends the “ESP header, creating “enchilada”.
- ❖ creates authentication MAC over the *whole enchilada*, using algorithm and key specified in SA;
- ❖ appends MAC to back of enchilada, forming *payload*;
- ❖ creates brand new IP header, with all the classic

Inside the enchilada:



- ❖ ESP trailer: Padding for block ciphers
- ❖ ESP header:
 - SPI, so receiving entity knows what to do
 - Sequence number, to thwart replay attacks
- ❖ MAC in ESP auth field is created with shared secret key

IPsec sequence numbers

- ❖ for new SA, sender initializes seq. # to 0
- ❖ each time dat _____ on SA:
- ❖ goal:
 - prevent attacker from sniffing and replaying a packet
 - receipt of duplicate, authenticated IP packets may disrupt service
- ❖ method:
 - destination checks for duplicates

Security Policy Database (SPD)

- ❖ policy: For a given datagram, sending entity needs to know if it should use IPsec
 - ma
address; protocol number
- ❖ info in SPD indicates “what” to do with arriving datagram
- ❖ info in SAD indicates “how” to do it

Summary: IPsec services



❖ suppose Trudy is eavesdropping on the communication between Alice and Bob

- will Trudy be able to determine the contents of the datagram? How about source, dest IP address, transport protocol, application port?
- flip bits without detection?
- masquerade as R1 using R1's IP address?
- replay a datagram?

IKE: Internet Key Exchange

- ❖ *previous examples:* manual establishment of IPsec SAs in IPsec endpoints:

Example SA

SPI: 12345

Encryption algorithm: 3DES-cbc

HMAC algorithm: MD5

Encryption key: 0x7aeaca...

HMAC key: 0xc0291f...

- ❖ manual keying is impractical for VPN with 100s of endpoints
- ❖ instead use *IPsec IKE (Internet Key Exchange)*

IKE: PSK and PKI

- ❖ authentication (prove who you are) with either
 - pre-shared secret (PSK) or
 - with PKI (public key and certificates).
- run IKE to generate IPsec SAs (one in each direction), including encryption, authentication keys
- ❖ PKI: both sides start with public/private key pair, certificate
 - run IKE to authenticate each other, obtain IPsec SAs (one in each direction).
 - similar with handshake in SSL.

IKE phases

- ❖ IKE has two phases
 - *phase 1*: establish bi-directional IKE SA
 - note: IKE IPsec SA

- ne
- ❖ phase 1 has two modes: aggressive mode and main mode
 - aggressive mode uses fewer messages
 - main mode provides identity protection and is more flexible

IPsec summary

- ❖ IKE message exchange for algorithms, secret keys, SPI numbers



au

- ESP protocol (with AH) additionally provides encryption
- ❖ IPsec peers can be two end systems, two routers/firewalls, or a router/firewall and an end system

Chapter 8 roadmap

8.1 What is network security?

8.2 Principles of cryptography

8.5 Securing TCP connections: SSL

8.6 Network layer security: IPsec

8.7 Securing wireless LANs

8.8 Operational security: firewalls and
IDS

WEP design goals



- ❖ symmetric key crypto

- confidentiality
- end host aut



- ❖ self-s

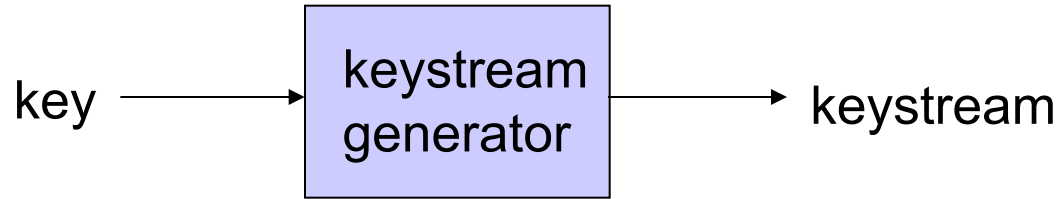
separately encrypted

- given encrypted packet and key, can decrypt; can continue to decrypt packets when preceding packet was lost (unlike Cipher Block Chaining (CBC) in block ciphers)

- ❖ Efficient

- implementable in hardware or software

Review: symmetric stream ciphers



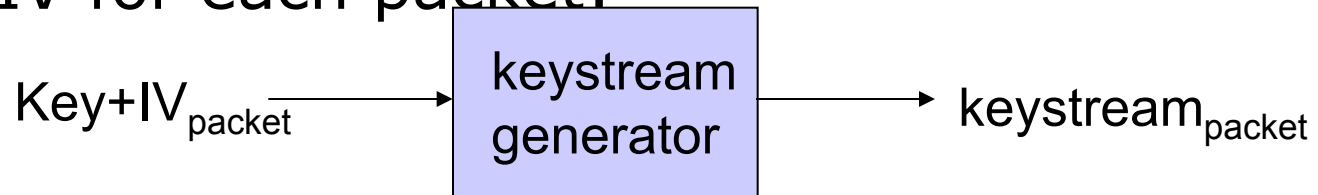
- $m(i)$ = i th unit of message
 - $ks(i)$ = i th unit of keystream
 - $c(i)$ = i th unit of ciphertext
 - $c(i) = ks(i) \oplus m(i)$ ($\oplus$ = exclusive or)
 - $m(i) = ks(i) \oplus c(i)$
- ❖ WEP uses RC4

Stream cipher and packet independence

- ❖ recall design goal: each packet separately encrypted
- ❖ if for frame n from where we

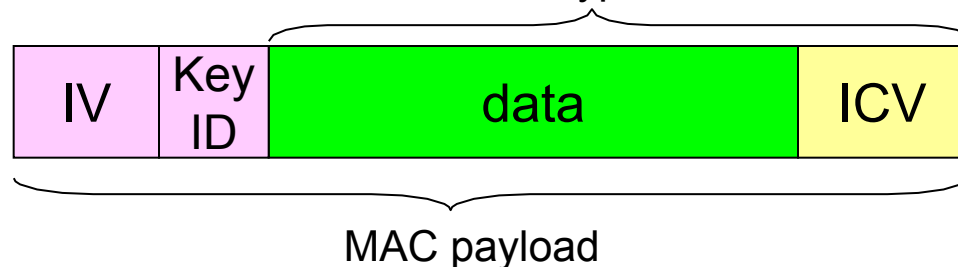
■ need ket n

- ❖ WEP approach: initialize keystream with key + new IV for each packet:

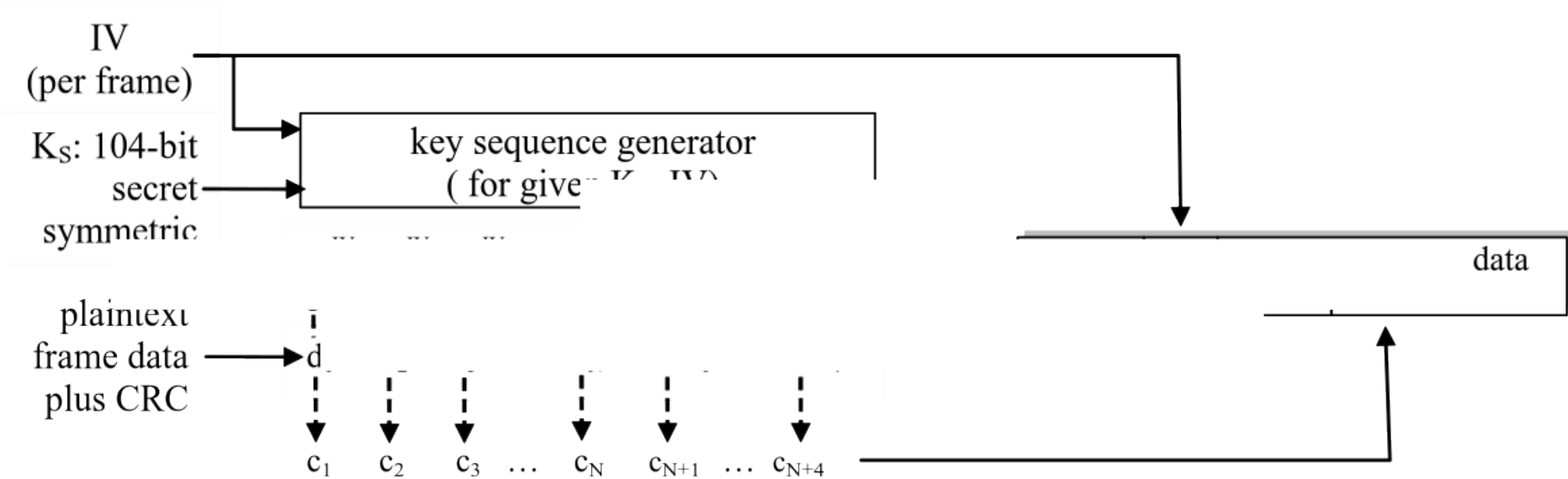


WEP encryption (1)

- ❖ sender calculates Integrity Check Value (ICV) over data
 - four-byte hash/CRC for data integrity
- ❖ each side has 104-bit shared key
- ❖ sender creates 24-bit initialization vector (IV), appends to key: gives 128-bit key
- ❖ sender also appends (ICV)
- ❖ data in
 - B\bytes of keystream are XORed with bytes of data & ICV
 - IV & keyID are appended to encrypted data to create payload
 - payload inserted into 802.11 frame

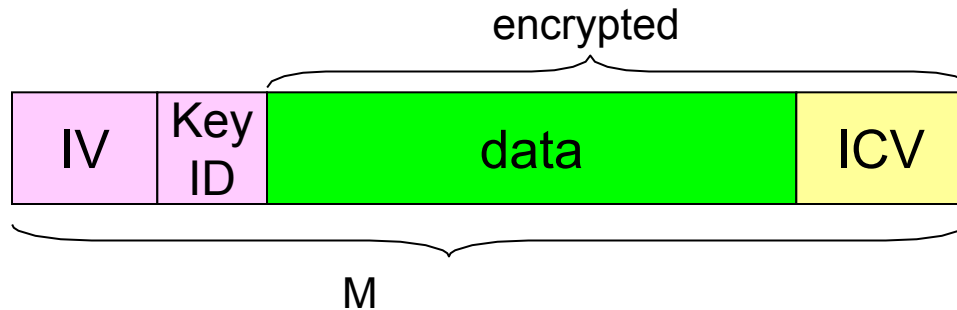


WEP encryption (2)



new IV for each frame

WEP decryption overview

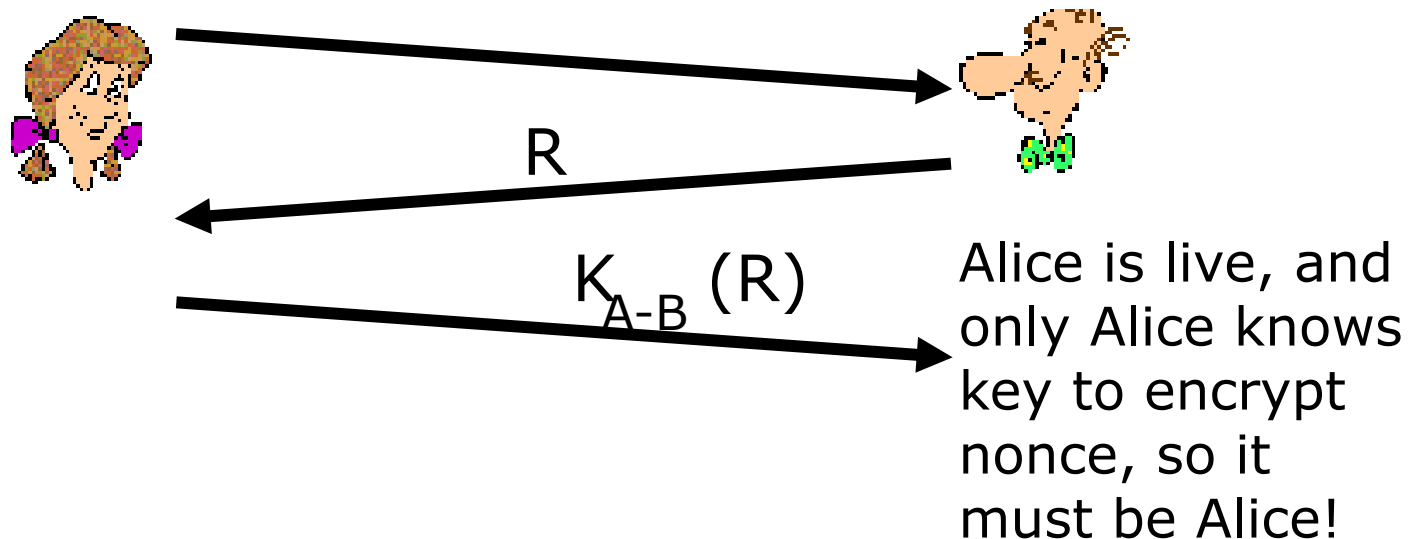


- ❖
 - ❖ rando
 - ❖ XORs keystream with encrypted data to decrypt data + ICV
 - ❖ verifies integrity of data with ICV
 - note: message integrity approach used here is different from MAC (message authentication code) and signatures (using PKI).

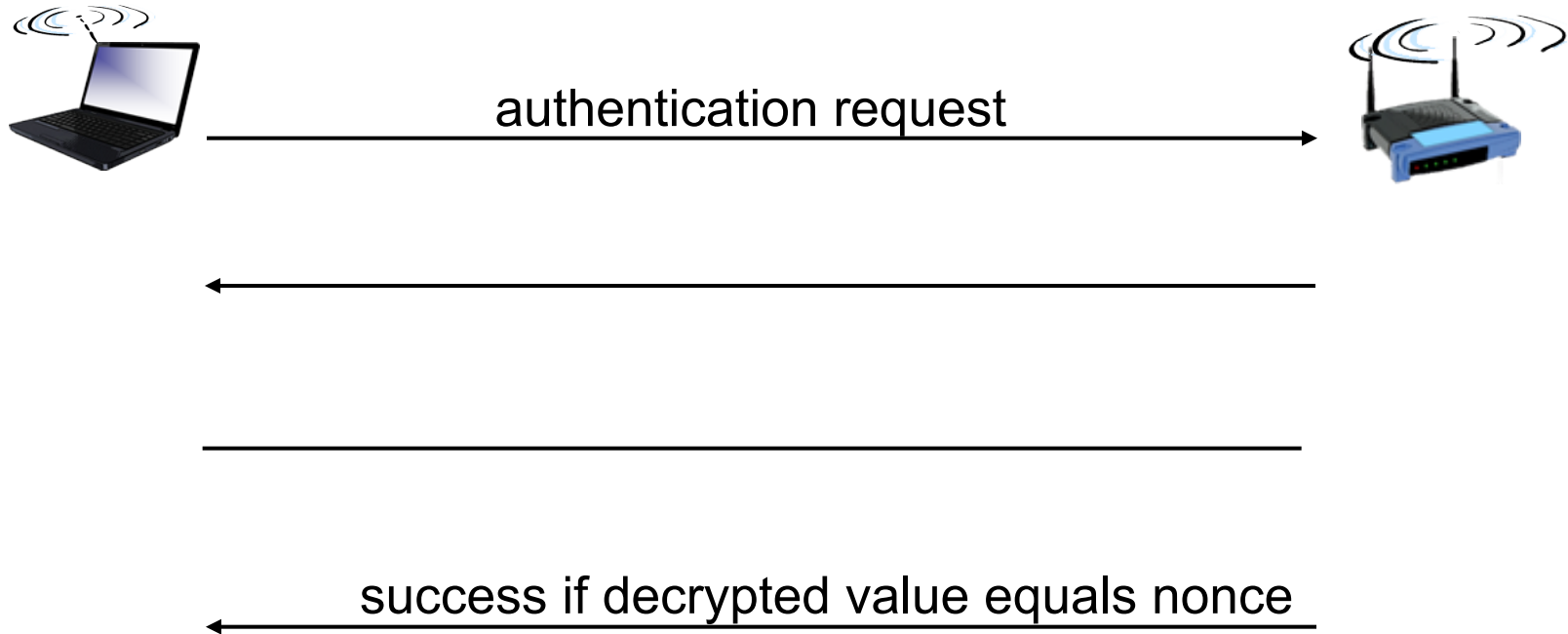
End-point authentication w/ nonce

nonce: number (R) used only *once* –*in-a-lifetime*

How to prove Alice "live": Bob sends Alice *nonce*, R. Alice



WEP authentication



Notes:

- ❖ not all APs do it, even if WEP is being used
- ❖ AP indicates if authentication is necessary in beacon frame
- ❖ done before association

Breaking 802.11 WEP encryption

security hole:

- ❖ 24-bit IV, one IV per frame, -> IV's eventually reused

❖

a

- Trudy causes Alice to encrypt known plaintext d_1 d_2 d_3 d_4 ...
- Trudy sees: $c_i = d_i \text{ XOR } k_i^{\text{IV}}$
- Trudy knows c_i d_i , so can compute k_i^{IV}
- Trudy knows encrypting key sequence k_1^{IV} k_2^{IV} k_3^{IV} ...
- Next time IV is used, Trudy can decrypt!

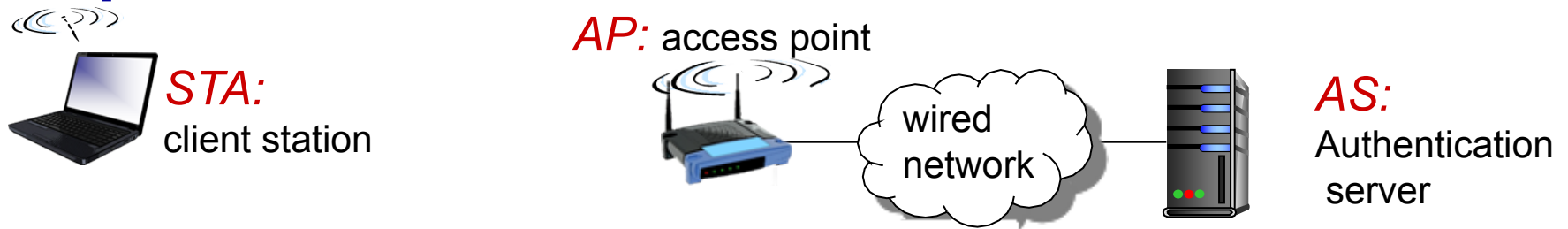
802.11i: improved ~~security~~

- ❖ numerous (stronger) forms of encryption possible



from

802.11i: four phases of operation



←—————┐————→
② STA and AS mutually authenticate, together generate Master Key (MK). *AP serves as “pass through”*

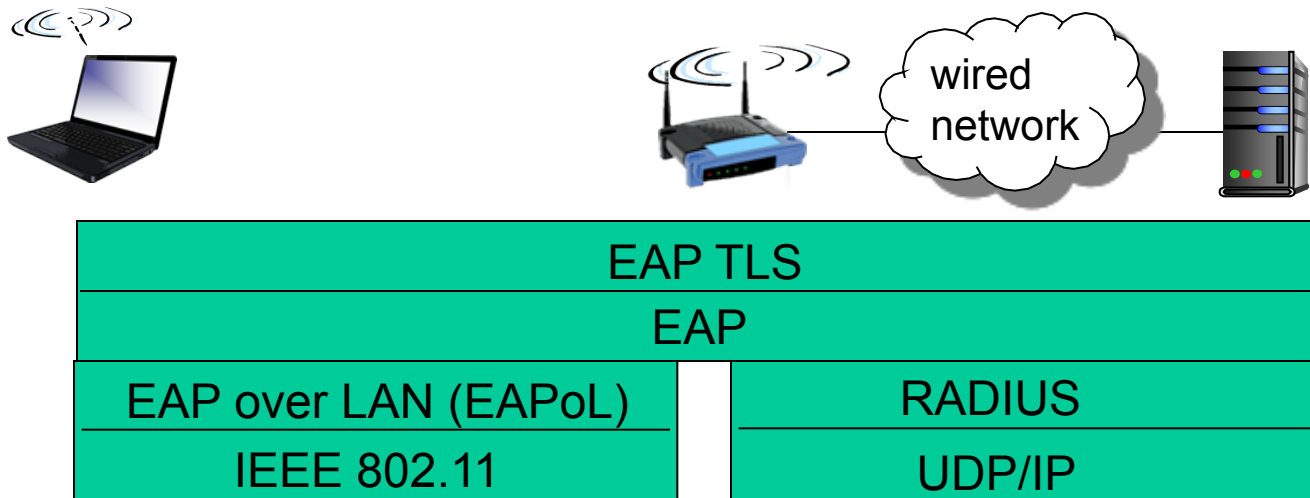
③ STA derives Pairwise Master Key (PMK)

←————→
③ AS derives same PMK, sends to AP

←————→
④ STA, AP use PMK to derive Temporal Key (TK) used for message encryption, integrity

EAP: extensible authentication protocol

- ❖ EAP: end-end client (mobile) to authentication server protocol
- ❖ EAP sent over ks''



Chapter 8 roadmap

8.1 What is network security?

8.2 Principles of cryptography

8.5 Securing TCP connections: SSL

8.6 Network layer security: IPsec

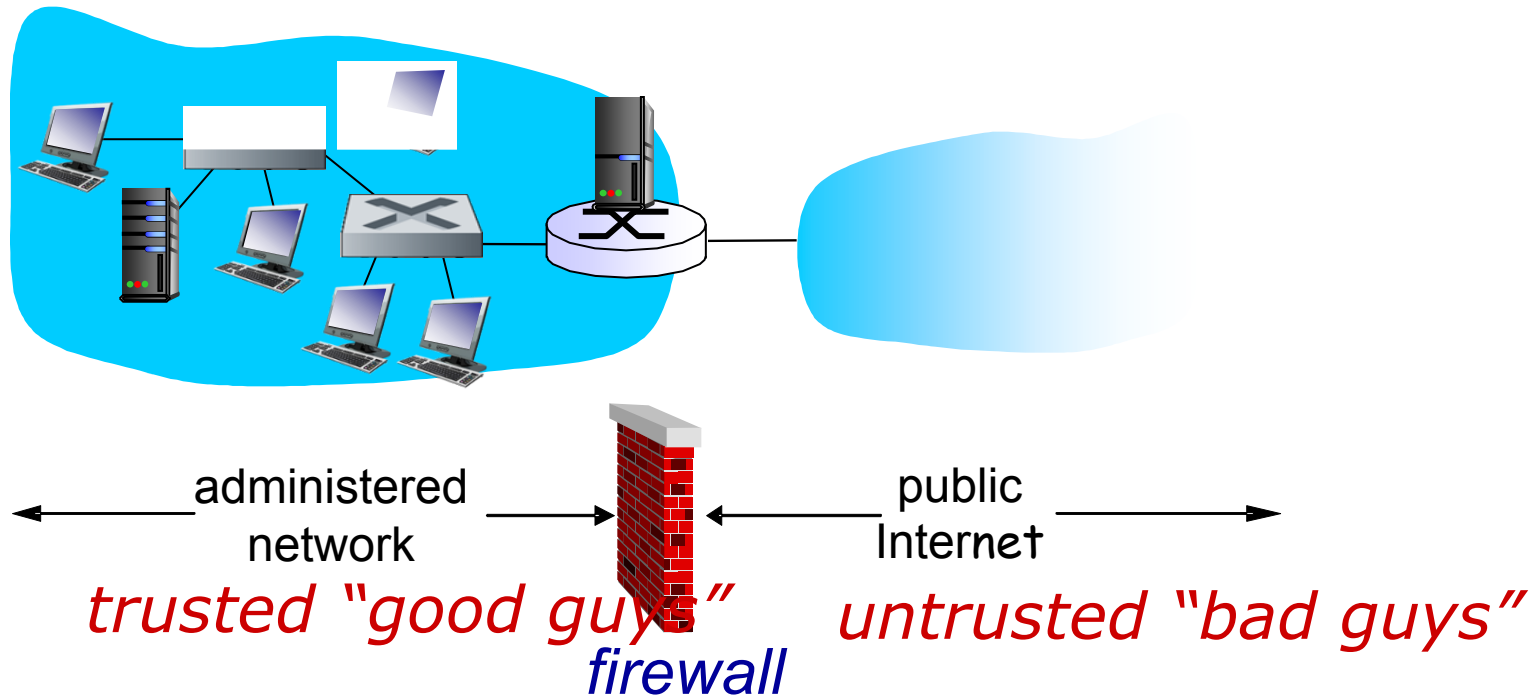
8.7 Securing wireless LANs

8.8 Operational security: firewalls and IDS

Firewalls

firewall

isolates organization's internal net from larger Internet, allowing some packets to



Firewalls: why

prevent denial of service attacks:

- ❖ SYN flooding: attacker establishes many bogus TCP connections, no resources left for “real” connections

p

- ❖ e.g., something else with

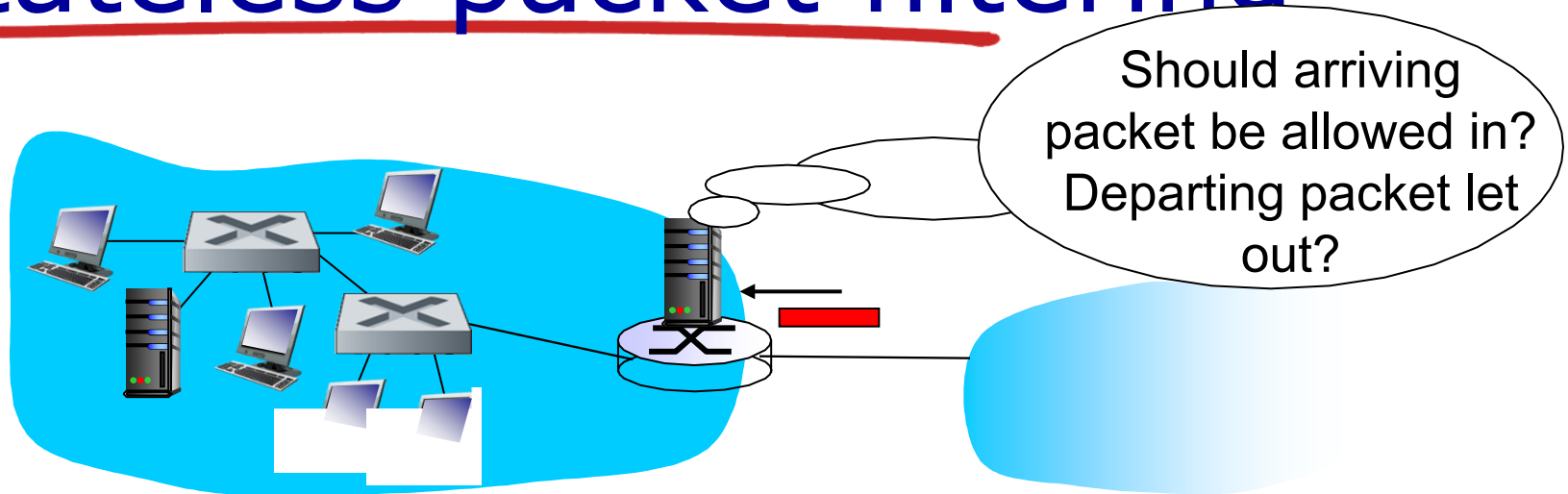
allow only authorized access to inside network

- ❖ set of authenticated users/hosts

three types of firewalls:

- ❖ stateless packet filters
- ❖ stateful packet filters
- ❖ application gateways

Stateless packet filtering



- ❖ internal network connected to Internet via *router firewall*
- ❖ router *filters packet-by-packet*, decision to forward/drop packet based on:
 - source IP address, destination IP address
 - TCP/UDP source and destination port numbers
 - ICMP message type
 - TCP SYN and ACK bits

Stateless packet filtering: ~~example~~

- ❖ *example 1:* block incoming and outgoing datagrams with IP protocol field = 17 and with either sou = 23

- ❖ *exa* with
ACK=0.

- *result:* prevents external clients from making TCP connections with internal clients, but allows internal clients to connect to outside.

Stateless packet filtering: more examples

<i>Policy</i>	<i>Firewall Setting</i>
No outside Web access.	Drop all outgoing packets to any IP address, port 80
public We	
Prevent Web-radios from eating up the available bandwidth.	Drop all incoming UDP packets - except DNS and router broadcasts.
Prevent your network from being used for a smurf DoS attack.	Drop all ICMP packets going to a "broadcast" address (e.g. 130.207.255.255).
Prevent your network from being tracerouted	Drop all outgoing ICMP TTL expired traffic

Access Control Lists

❖ **ACL:** table of rules, applied top to bottom to incoming packets: (action, condition) pairs

action	source			ource	dest	flag
allow	2					
allow	outside of 222.22/16	222.22/16	TCP	80	> 1023	ACK
allow	222.22/16	outside of 222.22/16	UDP	> 1023	53	---
allow	outside of 222.22/16	222.22/16	UDP	53	> 1023	----
deny	all	all	all	all	all	all

Stateful packet filtering

- ❖ *stateless packet filter*: heavy handed tool
 - admits packets that “make no sense,” e.g., dest port = 80, ACK bit set, even though no TCP connectio

allow	222.22/16					ACK

- ❖ *stateful packet filter*: track status of every TCP connection
 - track connection setup (SYN), teardown (FIN): determine whether incoming, outgoing packets “makes sense”
 - timeout inactive connections at firewall: no longer admit packets

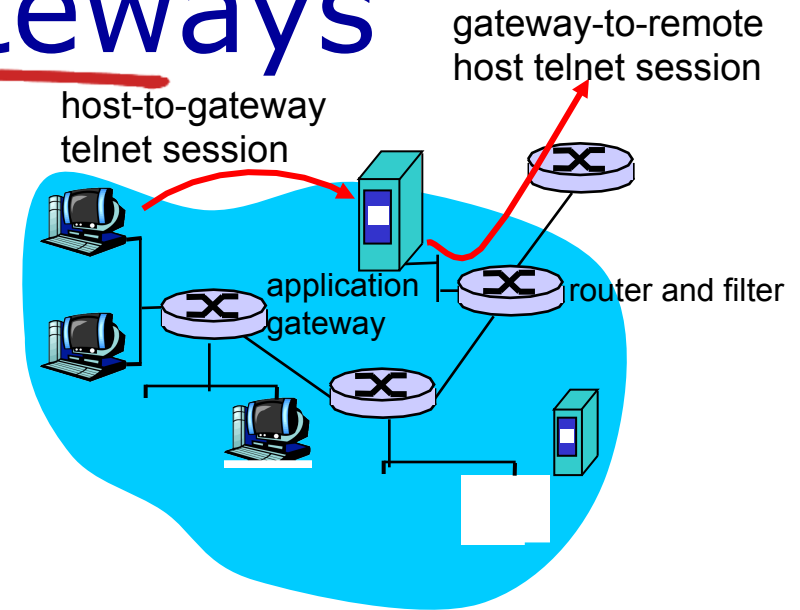
Stateful packet filtering

- ❖ ACL augmented to indicate need to check connection state table before admitting packet

action	source				dest	flag	check xion
allow	outside of 222.22/16	222.22/16	TCP	80	> 1023	ACK	X
allow	222.22/16	outside of 222.22/16	UDP	> 1023	53	---	
allow	outside of 222.22/16	222.22/16	UDP	53	> 1023	----	X
deny	all	all	all	all	all	all	

Application gateways

- ❖ filters packets on application data as well as on IP/TCP/UD

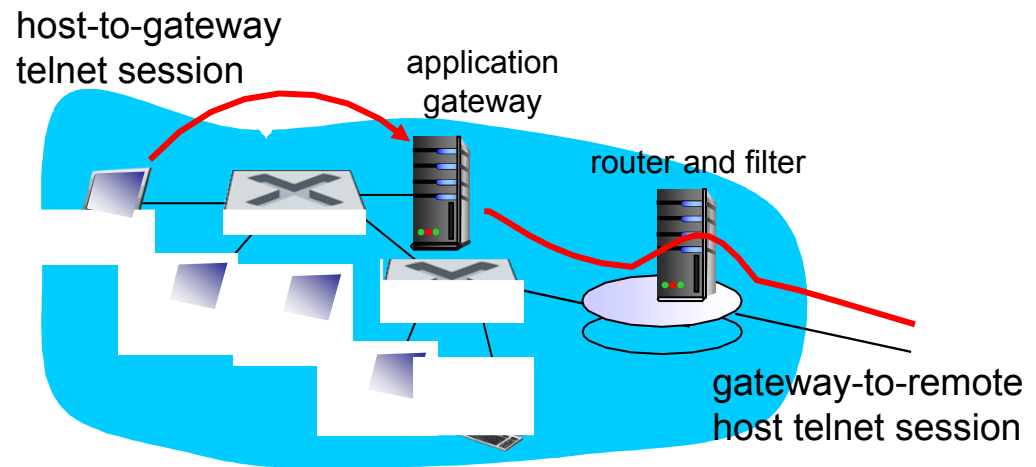


- ❖ internal
outside.
 1. require all telnet users to telnet through gateway.
 2. for authorized users, gateway sets up telnet connection to dest host. Gateway relays data between 2 connections
 3. router filter blocks all telnet connections not originating from gateway.

Application gateways

- ❖ filter packets on application data as well as on IP/TCP/UDP field

- ❖ internal telnet outside



1. require all telnet users to telnet through gateway.
2. for authorized users, gateway sets up telnet connection to dest host. Gateway relays data between 2 connections
3. router filter blocks all telnet connections not originating from gateway

Limitations of firewalls, gateways

- ❖ *IP spoofing*: router can't know if data "really" comes from

need
treatment, each has
own app. gateway

- ❖ client software must know how to contact gateway.
 - e.g., must set IP address of proxy in Web browser

- ❖ filters often use all or nothing policy for

level
of security

- ❖ many highly protected sites still suffer from attacks

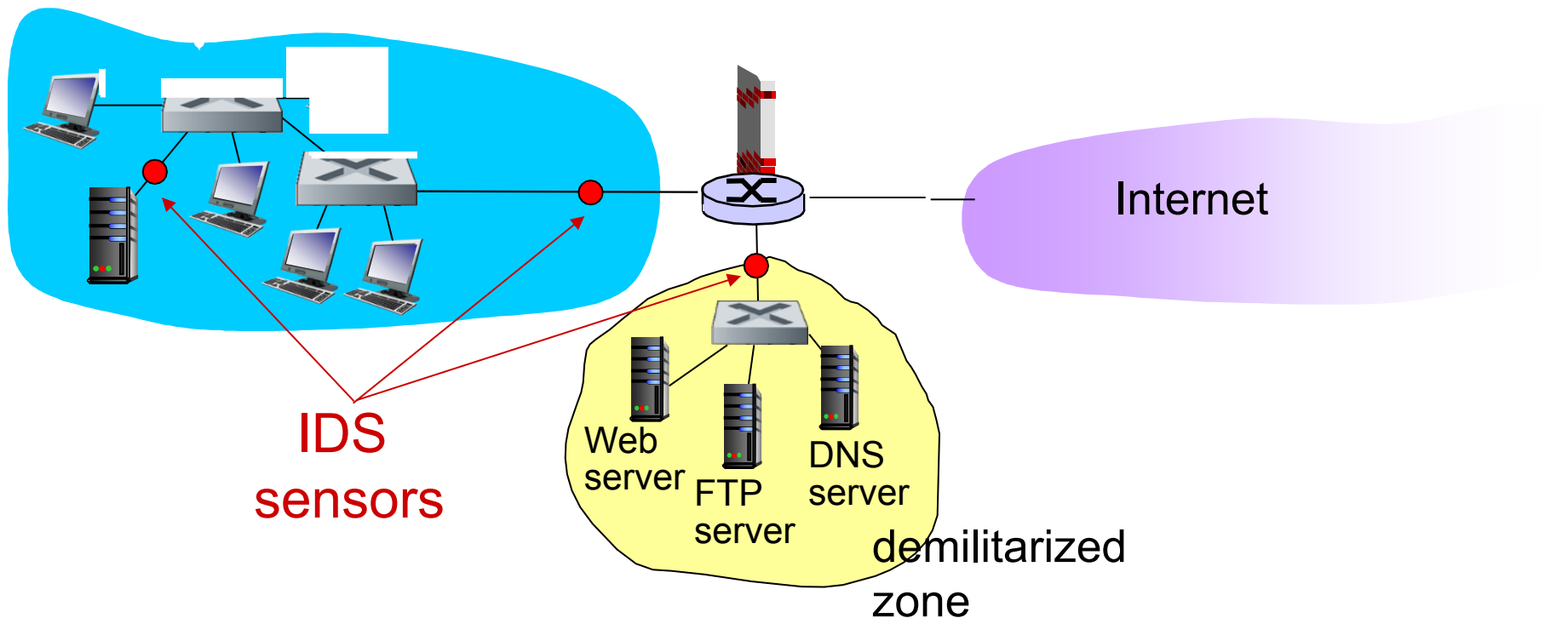
Intrusion detection ~~systems~~

❖ packet filtering:

- operates on TCP/IP headers only
- no correlation sessions
- *dee*
contents (e.g., check character strings in packet against database of known virus, attack strings)
- **examine correlation** among multiple packets
 - port scanning
 - network mapping
 - DoS attack

Intrusion detection ~~systems~~

- ❖ multiple IDSs: different types of checking at different locations



Network Security

(summary)

basic techniques.....

- cryptography (symmetric and public)
- message int
- secure email
- secure transport (SSL)
- IP sec
- 802.11

operational security: firewalls and IDS